JAVA

Dr. Meryem Janati Idrissi

Master Sciences du Numérique et Intelligence Artficielle
Université Mohammed V de Rabat
Année Universitaire : 2024/2025

Ce cours est fait pour vous,

n'hésitez pas à intervenir et poser vos questions quand vous le souhaitez !

Comment Se Déroulera Ce Module ?

- 10 séances
- Parties pratiques et moments de discussion
- Contrôle continue (participation)
- Examen final

Introduction générale

Introduction JAVA

Les environnements JAVA

IDE

Programmation procédurale

- La **programmation procédurale** est un paradigme de programmation qui repose sur l'utilisation de procédures (ou fonctions) pour structurer le code.
- Elle est basée sur la logique séquentielle et l'exécution d'instructions pas à pas.
- C'est une méthode qui procède à une analyse descendante pour résoudre un problème, en décomposant le problème en sous-problèmes jusqu'à ce que des actions très simples.

On a donc pensé aux traitements et à leur enchaînement AVANT de penser aux données → Approche « TOP-DOWN »

❌ Problèmes en cas de modification de la structures des données.
❌ Développer un très grand programme en procédural peut donc s'avérer long et fastidieux.
❌ Les données et les fonctions sont séparées, ce qui peut rendre le code moins sécurisé et plus difficile à maintenir.

Programmation orienté objet

- La **programmation orientée objet**, ou POO, est un modèle de programmation qui repose sur le concept de classes et d'objets.
- Elle est utilisée pour structurer un programme logiciel en éléments de code simples et réutilisables, qui sont utilisés pour créer des instances individuelles d'objets.
- Un objet est tout ce qui peut être nommé et décrit.
- Les objets sont des « choses » qui sont modélisées dans des programmes et, par conséquent, le but d'un programme orienté objet est de représenter le monde réel en code.
  - Etudier les traitements comme des interactions entre les différentes entités.

Penser aux données AVANT de penser aux traitements.

  - Penser à un programme en modélisant le monde tel qu'il nous apparaît.

Historique de la POO

- **Les années 60**: les premiers pas de la programmation orientée objet
  - Simula-67, langage de simulation informatique
- **Les années 70**: apparition des concepts de base
  - objet, encapsulation, polymorphisme, héritage, ...
- **Les années 80**: explosion de l'orientée objet
  - Objective C, C++, Eiffel, ...
- **Les années 90**: l'âge d'or de la POO
  - Standardisation de C++
  - Apparition du langage Java

Java

Size of programming language communities in Q1 2024

		Most popular in	Least popular in
JavaScript*	25.2 M	Web	AR/VR
Python	18.2 M	Machine Learning/AI	Mobile
Java	17.7 M	Backend Services	AR/VR
C++	11.6 M	Embedded	Web
C#	10.2 M	Desktop	Machine learning/AI
PHP	9.8 M	Web	Machine learning/AI
Visual development tools	7.2 M	AR/VR	Backend services
C	6.5 M	Embedded	Web
Kotlin	5.6 M	Mobile	Web
Go	4.7 M	Backend Services	Mobile
Swift	4.6 M	Mobile	Backend services
Rust	4.0 M	Embedded	Mobile
Dart	2.9 M	Mobile	Web
Objective-C	2.7 M	On-device (consumer electronics)	Desktop
Ruby	2.5 M	On-device (consumer electronics)	Web
Lua	1.8 M	On-device (consumer electronics)	Mobile

Les données ont été tirées de la Developer Research Report: Sizing programming language communities

Java

- Java est un langage compilé et interprété :

  Compilé :

  - Le code source Java (.java) est compilé en bytecode (.class) par le compilateur Java (javac).
  - Ce bytecode est un code intermédiaire qui n'est pas directement exécutable par le processeur.

  Interprété :

  - Le bytecode est interprété et exécuté par la Java Virtual Machine (JVM), qui le traduit en instructions machine spécifiques au système d'exploitation.

- Ce qui signifie qu'un programme java n'est pas directement exécutable par le système d'exploitation mais il doit être interprété par un autre programme.

Java

Source Java

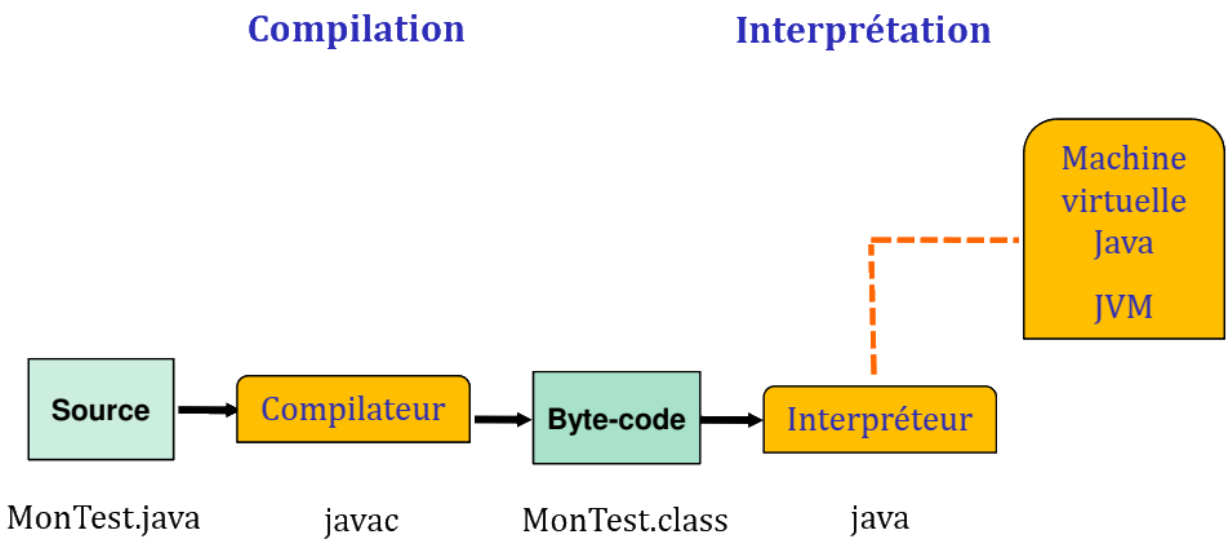
Fichier utilisé lors de la phase de programmation.
Le seul fichier réellement intelligible par le programmeur.

Byte-code Java

Code objet destiné à être exécuté sur toute « Machine virtuelle » Java
Provient de la compilation du code source.

Machines Virtuelles Java

Des interpréteurs de byte-code indépendants (pour exécuter les programmes Java).
Contenues au sein d'un navigateur (pour exécuter des applets Java).



Java

On écrit un programme java sauvegardé dans un fichier d'extension .java (exemple Test01.java : supposé contenir une classe nomée Test01)

On compile ensuite le programme pour générer le Byte
Code associé :

```
C:\ … > javac Test01.java
```

Un fichier Test01.class est alors créé.

```
C:\ … > java Test01
```

Caractéristiques de java

Simple et Facile à Apprendre

- On peut l'apprendre facilement :
  - Peu de mots-clés
  - Facilité des fonctionnalités principales.
- Se base sur les concepts fondamentaux des langages C et C++: En reprenant une grande partie de la syntaxe de ces deux langages, Java est vite pris en main par les développeur C et C++.

Orienté Objet (POO)

- Java suit les principes de la programmation orientée objet (POO) :
  - Encapsulation : Protéger les données avec des classes et des modificateurs (private, public).
  - Héritage : Réutilisation du code via des classes parentes et enfants.
  - Polymorphisme : Une même méthode peut s'adapter à plusieurs types d'objets.
  - Abstraction : Cacher les détails de l'implémentation avec des classes abstraites et interfaces.

Caractéristiques de java

Robuste et Fiable

- Gestion automatique de la mémoire avec Garbage Collector.
- Gestion avancée des erreurs avec try-catch-finally (Exceptions).
- Forte vérification des types à la compilation et à l'exécution.

Indépendant de la plateforme

- Grâce à la JVM (Java Virtual Machine), Java est portable ("Write Once, Run Anywhere").
- Un programme compilé en bytecode (.class) peut s’exécuter sur Windows, Linux, Mac, Android…
- Il n'est pas lié à un système d'exploitation spécifique.

Caractéristiques de java

Langage multi-tâches

- Exécution de plusieurs processus effectuant chacun une tâche différente simultanément.
- Mécanismes de synchronisation.
- Fonctionnement sur des machines multiprocesseurs.

Langage multi-thread

- Permet l'exécution parallèle de plusieurs tâches grâce aux threads.
- Un thread est une portion de code capable de s'exécuter en parallèle à d'autres traitements.
- Exécution de plusieurs threads d'un processus simultanément

Caractéristiques de java

Évolutif et Riche en Bibliothèques

- API riche (java.util, java.io, java.sql, java.net, javax.swing...).
- Possède un large écosystème : Spring, Hibernate, Jakarta EE, JavaFX, Android...
- Supporte les bases de données (JDBC, JPA) et le développement Web (Servlets, JSP, Spring Boot).

Sécurisé

- Des fonctions de sécurité intégrées au langage et au système d'exécution:
  - La vérification du bytecode pour le code douteux,
  - la prise en charge de l'authentification et de la confidentialité, etc.

Introduction générale

Introduction JAVA

Les environnements JAVA

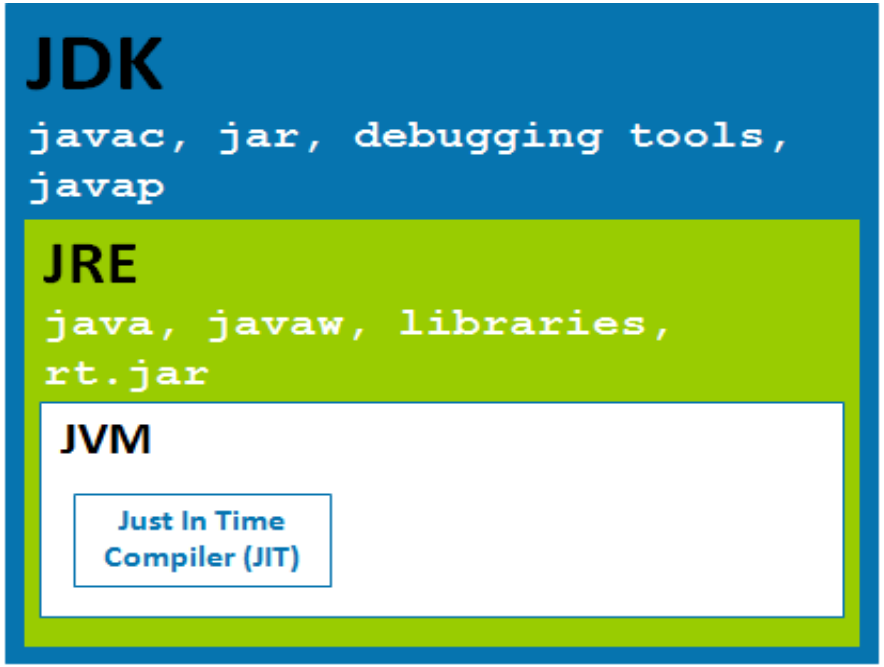
IDE

Environnement Java

Pour comprendre les bases de Java, il est essentiel de se familiariser avec des concepts clés tels que :

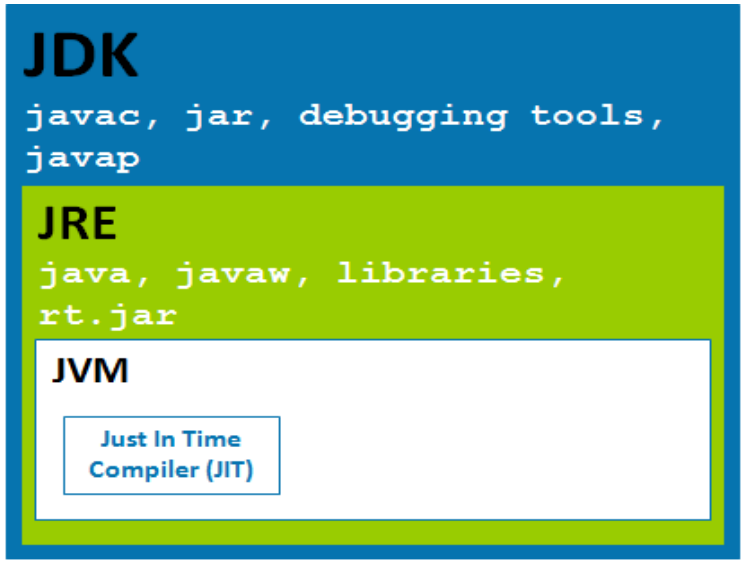
- le Java Development Kit (JDK),
- le Java Runtime Environment (JRE),
- la Java Virtual Machine (JVM).

Ce sont trois composants indispensables de la plate-forme Java.
Ils fonctionnent ensemble dans les applications Java.



Java Development Kit (JDK)

- Le kit de développement Java (JDK) est un ensemble d'outils essentiels pour le développement d'applications Java.
- Il permet de fournir un environnement complet pour écrire, compiler, déboguer et exécuter des programmes Java.
- Les outils fournis par le JDK sont généralement rencontrés dans tous les environnements de développement logiciel.



Java Development Kit (JDK)

Les composants du JDK

Le compilateur Java (javac): joue un rôle essentiel dans ce processus. Il prend le code source Java et le convertit en bytecode Java compréhensible par la JVM. Le compilateur vérifie la syntaxe, les types et effectue des optimisations lors de la compilation.

Les bibliothèques standard (Java Class Library): Le JDK inclut une vaste bibliothèque de classes prédéfinies, appelées la Java Class Library.

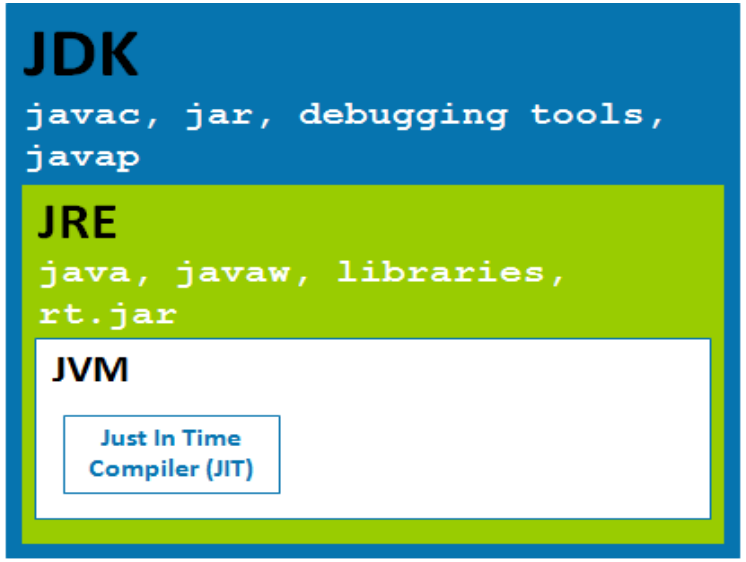
- Ces bibliothèques offrent un large éventail de fonctionnalités prêtes à l'emploi, telles que la gestion des chaînes, les opérations sur les fichiers, les interfaces graphiques, les connexions réseau, etc.

Les outils de développement: Le JDK propose une gamme d'outils de développement pour faciliter le processus de création d'applications Java.

- Parmi ces outils figurent le débogueur (jdb) pour identifier et corriger les erreurs ou encore le générateur de documentation (javadoc) pour créer une documentation lisible par les développeurs.

Java runtime Environment (JRE)

- Le Java Runtime Environment (JRE) est un environnement d'exécution nécessaire pour exécuter des applications Java.
- Il comprend la Java Virtual Machine (JVM), les bibliothèques de classes et d'autres composants essentiels.
- Il prend en charge l'exécution des programmes Java en fournissant une plateforme d'exécution pour la JVM.
- Il charge et exécute les fichiers bytecodes Java et fournit l'accès aux bibliothèques de classes nécessaires.



Java Runtime Environment (JRE)

Différences entre le JDK et le JRE :

- Le JDK comprend à la fois le JRE et les outils de développement, tandis que le JRE est destiné à l'exécution des applications Java.
- Le JDK est utilisé pour le développement, tandis que le JRE est utilisé pour l'exécution des applications Java.

Importance du JRE :

- Le JRE est crucial car il garantit la portabilité des applications Java.
- Les applications Java peuvent être exécutées sur n'importe quel système compatible avec le JRE, indépendamment du système d'exploitation sous-jacent, tandis que la JVM est disponible.

Java Virtual Machine (JVM)

- La Java Virtual Machine (JVM) est un composant essentiel de l'environnement d'exécution Java.
- Son rôle principal est d'exécuter les programmes Java en convertissant le bytecode Java en instructions exécutables pour la machine sous-jacente.
- La JVM permet la portabilité des programmes Java, ce qui signifie qu'ils peuvent être exécutés sur différentes plates-formes sans nécessiter de modifications majeures.
- Elle fournit une couche d'abstraction entre le code Java et le système d'exploitation, permettant aux programmes Java d'être indépendants de la plateforme.

API Java

- Java fournit de nombreuses bibliothèques de classes remplissant des fonctionnalités très diverses : c'est l'API Java.
- **L'API Java (Application Programming Interface)** est une bibliothèque standard fournie avec Java. Elle contient un ensemble de classes, interfaces et méthodes prêtes à l'emploi pour simplifier le développement d'applications Java.
- Elle permet aux développeurs d'utiliser des fonctionnalités sans avoir à tout programmer depuis zéro.
- L'API Java est organisée en packages, qui regroupent des classes liées par leur fonctionnalité.

API Java

Les principaux paquetages :

java.util : structures de données classiques

java.io : entrées / sorties

java.lang : chaînes de caractères, interaction avec l'OS, threads

java.awt : interfaces graphiques, images et dessins

javax.swing : package récent proposant des composants « légers » pour la création d'interfaces graphiques

java.net : sockets, URL

java.sql : fournit le package JDBC

Introduction générale

Introduction JAVA

Les environnements JAVA

IDE

IDE Java

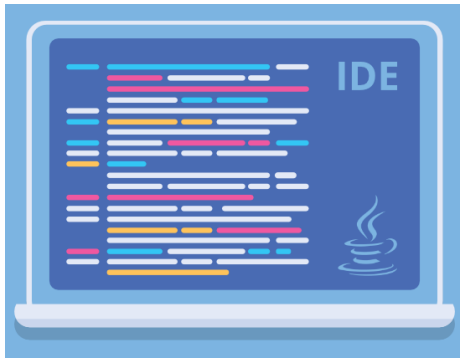
- Un **IDE (Integrated Development Environment)** ou environnement de développement intégré se définit comme une application logicielle rassemblant au même endroit tous les outils requis pour un projet de développement logiciel.
- Il dispose d'une interface permettant d'écrire du code, de gérer des groupes de texte, et d'automatiser les tâches redondantes de programmation.
- Les IDE rassemblent toutes les fonctionnalités de différents processus de programmation en un seul : un éditeur, un compilateur, un débogueur…etc.
- Avec les IDE, les développeurs gagnent plus de temps dans leur programmation, notamment dans l'écriture de code.
- En effet, ils n'ont plus besoin d'apprendre à utiliser ces outils séparément.

IDE Java

L'IDE: *Integrated Development Environment*.

Il regroupe un ensemble d'outils spécifiques dédiés aux programmeurs pour faciliter la mise en œuvre de projets tels que le développement de logiciels..

Eclipse est un logiciel qui va nous permettre de développer nos applications ou nos applets, et aussi celui qui va compiler tout ça. Notre logiciel va donc permettre de traduire nos futurs programmes Java en byte code, compréhensible uniquement par votre JRE,



Les variables et les opérateurs

```
<Type de la variable> <Nom de la variable> ;
```

En Java, nous avons deux types de variables :

1- des variables de type simple ou « primitif » ;

2- des variables de type complexe ou des « objets ».

types simples ou types primitifs, en Java, ce sont des nombres entiers, des nombres réels, des booléens ou encore des caractères

Les variables et les opérateurs

Les variables de type numérique

Afin d'informer la JVM que le type utilisé est long, vous DEVEZ ajouter un "L" à la fin de votre nombre, sinon le compilateur essaiera d'allouer ce dernier dans une taille d'espace mémoire de type entier et votre code ne compilera pas si votre nombre est trop grand...

Vous remarquerez que nous ne mettons pas une virgule, mais un point ! Et vous remarquerez aussi que même si le nombre en question est rond, on écrit « .0 » derrière celui-ci, le tout suivi de « f ».

byte	1 octet	-128 et +127	byte temperature; temperature = 64;
short	2 octets	-32768 et +32767	short vitesseMax; vitesseMax = 32000;
int	4 octets	-2*10^9 à 2*10^9	int temperatureSoleil; temperatureSoleil = 15600000;
long	8 octets	-9*10^18 à 9*10^18	long anneeLumiere; anneeLumiere = 9460700000000000**L**;
float	4 octets	$\pm$1.4*10^-45 à $\pm$ 3.4 $*$ 10^38	float pi; pi = 3.141592653**f**;
			float nombre; nombre = 2.0f;
double	8 octets	$-$1.79*10^308 à 1.79 $*$ 10^308	double division; division = 0.333333333333333333333333333333333333333333333334d;

Les variables et les opérateurs

- **Des variables stockant un caractère**

```
char caractere;
caractere = 'A';
```

- **Des variables de type booléen**

```
boolean question;
question = true;
```

```
System.out.println(« The value of a is : » + a);
```

Le type String

Il s'agit d'une variable d'un type plus complexe que l'on appelle objet.

Le type String permet de gérer les chaînes de caractères, c'est-à-dire le stockage de texte.

String n'est pas un type de variable, mais un objet. Notre variable est un objet, on parle aussi d'une instance : ici, une instance de la classe String.

```
//Première méthode de déclaration
String phrase; phrase =  « Java";
//Deuxième méthode de déclaration
String str = new String(); str = "Une autre chaîne de caractères";
//Troisième méthode de déclaration
String str2 = "Une autre chaîne";
//Quatrième méthode de déclaration
String chaine = new String("Et une de plus !");
```

Pourquoi **String** commence par une majuscule et pas les autres ?

Attention : String commence par une majuscule ! Et lors de l'initialisation, on utilise des guillemets doubles (« " " »).

Convention de nommage

✓tous vos noms de classes doivent commencer par une majuscule ;
✓tous vos noms de variables doivent commencer par une minuscule ;
✓si le nom d'une variable est composé de plusieurs mots, le premier commence par une minuscule, le ou les autres par une majuscule, et ce, sans séparation ;
✓tout ceci sans accentuation !

```
public class Classe{}
public class Nombre{}
String chaine;
String chaineDeCaracteres;
int nombre;
int nombrePlusGrand;
```

les variables de type **String** sont des **objets**. Les objets sont définis par une un squelette qui est en fait une **classe**. c'est pourquoi String a une majuscule et pas int, float, etc., qui eux ne sont pas définis par une classe.

Les conversions « cast »

D'un type int en type float :

```
int i = 123;
float j = (float)i;
```

D'un type int en double :

```
int i = 123;
double j = (double)i;
```

D'un type double en int :

```
double i = 1.23;
double j = 2.9999999;
Int k;
k = (int)i; //k vaut 1
k = (int)j; //k vaut 2
```

```
double nbre1 = 10, nbre2 = 3;
int resultat = (int)(nbre1 / nbre2); System.out.println("Le résultat est = " +
resultat);
```

```
int nbre1 = 3, nbre2 = 2;
double resultat = nbre1 / nbre2;
double resultat = (double)(nbre1 / nbre2);
double resultat = (double)(nbre1) / (double)(nbre2);
```

2-Structure de base

Structure d'une application java

Syntaxe de base et première application

Structure générale d'un programme Java

Le squelette général d'un programme Java se présente comme suit (chaque classe dans un fichier séparé et portant le même nom que celui de la classe):

```
public class No
        // Propriété
        // Méthodes
}
```

```
public class Nom
        // Propriétés
        // Méthodes
}
```

```
public class NomDeClassePrincipale {
        // Propriétés de la classe
        // Méthodes de la classe
        public static void main(String args[]) {
                // Code du programme principal
        }
}
```

Structure générale d‘un programme Java

Il est à remarquer que le compilateur Java génère un fichier de classe (.class) pour chacune des classes qui constituent le programme. Il est donc d’usage d’écrire chaque classe séparément (dans un fichier séparé portant le même nom que celui de la classe et avec respect de la casse). Le compilateur n’aura pas de problème pour retrouver la classe compilée (si elle est référencée à partir d’une autre classe qui se trouve dans un autre fichier) tant qu’on travaille dans le même répertoire (ou plus exactement dans le même package). Dans le cas contraire (différents packages), on utilise l’instruction **import** pour préciser le chemin des classes accessibles.

Structure générale d‘un programme Java

Example:

```java
//La classe est l'unité de base de nos programmes.
public class MyFirstProgram { //Accolade débutant la classe PremProg
    public static void main(String[] args)
    { //Accolade débutant la méthode main

        /* Pour l'instant juste une instruction.
            Ce commentaire s'étale sur plusieurs lignes */
        // Une instruction se termine par une ;

        System.out.println("Hi! This is my first program YAY!!");
    } //Accolade fermant la méthode main
}//Accolade fermant la classe PremProg
```

Structure générale d‘un programme Java

String[] args: permet de récupérer des arguments transmis au programme au moment de son lancement. On peut lancer un programme sans fournir d’arguments, mais l’indication String args[] est obligatoire.

```
public class NomDeClassePrincipale {

        // Propriétés de la classe

        // Méthodes de la classe

        public static void main(String args[]) {

                // Code du programme principal

        }

}
```

Structure générale d‘un programme Java

Example:

```
public class Hello {
    public static void main(String[] args) {
            sayHelloTo("Student X"); // sayHelloTo(args[0]) si on veut
        }
    // affiche le message "hello" à l'étudiant fourni
    private static void sayHelloTo(String nomEtudiant) {
        System.out.println("Hello " + nomEtudiant);
    }
}
```

Modificateurs et visibilité

Nous remarquons que la méthode **main()** qui constitue le point d'entrée de toute application est précédée par 3 mots réservés **public static et void**. Le mot clé void désigne le type de retour, c'est donc une procédure. **public** et **static** sont des modificateurs. Il existe 5 types de modificateurs très utilisés qui peuvent être associées à une données ou une fonction membre : modificateurs de synchronisation, de visibilité, de permanence, de constance et d'abstraction.

1- Synchronisation	2
SYNCHRONIZED (seulement avec les méthodes)	**p - p p**
5- ABSTRACTION	
ABSTRACT (seulement les méthod **private**)	

```
public class NomDeClassePrincipale {

    // Propriétés de la classe

    // Méthodes de la classe

    public static void main(String args[]) {

        // Code du programme principal

    }
}
```

Type	Nom
void int …	

Modificateurs et visibilité

Exemples :

- **synchronized public static final** void p1() { … }
- **public abstract** void p2() { … }
- **public static int** f1() { … }

Modificateurs: synchronized, private, public et protected

Modificateur synchronized:

- Permet de mettre en place une méthode ou un bloc de programme verrouillé par l'intermédiaire du mécanisme de moniteur. Une méthode **synchronized** est une méthode dont l'accès sur un même objet est réalisé en exclusion mutuelle.

Modificateurs private, public et protected:

- Une donnée ou méthode **private** est inaccessible depuis l'extérieur de la classe où elle est définie même dans une classe dérivée.
- Une donnée ou méthode **public** est accessible depuis l'extérieur de la classe où elle est définie
- Une donnée ou méthode **protected** est protégée de tout accès externe comme private sauf à partir des classes dérivées. Une méthode ou donnée protected est donc accessible dans une classe fille.

Modificateurs: synchronized, private, public et protected

Remarque:

Lorsqu'on redéfinie une méthode dans une classe fille, il est obligatoire de conserver son modificateur de visibilité ou d'utiliser un privilège d'accès plus fort. Càd :

1) une méthode public reste public,
2) une méthode protected reste protected ou devient public,
3) et une méthode private reste private ou devient public ou protected.

Modificateur static

Modificateur static:

- Le modificateur **static** permet de définir une donnée ou une méthode commune à toutes les instances de la classe. Ce sont des entités qui existent en l'absence de tout objet de la classe. Ainsi, une donnée ou méthode static est associée à sa classe entière, et non à une instance particulière de la classe.
- Le mot clé static peut aussi être appliqué à toute la classe (static class mais uniquement pour les classes internes) dans tel cas tous les membres de la classe seront static.

Les membres **static** peuvent donc être appelés directement par l'intermédiaire du nom de la classe. On n'a pas besoin de créer une instance de la classe pour accéder à une méthode ou donnée static se trouvant dans cette classe. L'appel est réalisé simplement à l'aide du sélecteur << . >> >> (le C++, utilise pour ce cas le sélecteur << :: »).
Les données static sont appelées variable de classe et les méthodes static sont également appelées méthodes de classe. Un exemple concret de méthodes static est la méthode main() qui doit être static afin de rester indépendante des autres objets que le programme peut générer à partir de sa classe principale.

Modificateur static

Remarque:

Puisque les données non-static d'une classe n'ont d'existence que lorsqu'on a instancier un objet de la classe et inversement une méthode static existe sans avoir besoin de créer d'instance de classe, une méthode static ne peut donc pas accéder à une donnée non-static ou une autre méthode non-static:

Exemple :

```
class Class1 {
    static int x = 20;
    int y = 30;
    static void p1() {
        x = x + 1 ; // correcte
        y = y * 2 ; // incorrecte
        p2() ; // incorrecte
    }
    void p2() { // méthode non-static
        x ++ ; // correcte
        y ++ ; // correcte
        p1() ; // correcte
    }
}
```

Modificateur final

Les données **final** sont des constantes. Ces données sont généralement définies en plus public et static afin d'être accessibles depuis l'extérieur de la classe et directement par l'intermédiaire du nom de celle-ci sans avoir besoin de créer une instance de la classe.

Exemple :

```
class Constantes {
        public static final int CONST1 = 20;
        public static final double PI = 3.14;
}
```

Modificateur abstract

Ce modificateur permet de définir une méthode abstraite. Càd une méthode dont le corps n'est pas défini. Une classe qui comporte une méthode abstraite doit, elle aussi, être définie abstraite. Il s'agit d'une classe de spécification qui ne peut être directement instanciée. Elle nécessite d'être redéfinie dans une classe fille qui aura donc l'objectif d'implémenter les spécifications de la classe abstraite mère.
La définition de méthodes abstraites est réalisée de la manière suivante :

```
abstract class ClassAbstraite {

        ...

        abstract typeretour methodeAbstraite(parametres) ;

}
```

Modificateur abstract

Remarque:

Vue la notion d'abstraction, une méthode abstraite ne peut pas être :

- **synchronized** car elle n'est pas encore définie
- **static** pour la même raison (elle n'a pas encore d'existence, or une méthode static existe dès la création de la classe).
- **final** car cela va empêcher sa redéfinition, or une méthode abstract n'a d'existence que lorsqu'elle est défine
- **private** la redéfinition d'une méthode private signifie en fait la définition d'une nouvelle méthode qui porte le même nom (et non pas le véritable sens de la redéfinition). De ce fait on ne peut pas donner d'existence à une méthode abstraite déclarée private. Ce qui interdit la combinaison private-abstract.

Modificateurs avec la méthode main

La remarque que nous avons fait à propos des méthodes **static** reste valable pour le main qui est une méthode **static**. Cela impose une grande restriction, car la méthode main ne peut accéder qu'à des données et des méthodes static. Pour résoudre ce problème, il suffit de créer un objet de la classe principale dans la méthode main. Ce qui implique l'exécution du constructeur. Ce dernier n'est pas une méthode static on peut alors y mettre tous les accès nécessaires aux différents membres non-static. Le squelette d'un programme Java est souvent alors formé de la manière suivante :

```
class NomDeClasse {
        // données membres
        NomDeClasse ()  { // Constructeur
                ...
        }
        ...
        public static void main(String args[]) {
                new NomDeClasse();
        }
}
```

Modificateurs avec la méthode main

Exemple:

```
class Class1 {
    static int x = 20;
    int y = 30;
    Class1() {
        x = x + 1 ; // correcte
        y = y * 2 ; // correcte
        p2() ; // correcte
    }
    void p2() {
        System.out.println("x = " + x + "    y = " + y) ;
    }

    public static void main(String args[]) {
        new Class1();
        y = y * 2 ; // incorrecte
        p2() ; // incorrecte
    }
}
```

```java
class Class1 {
    static int x = 20;  // Variable statique (partagée par toutes les instances)
    int y = 30;         // Variable d'instance (propre à chaque objet)

    // Constructeur de la classe
    Class1() {
        x = x + 1;  // Correct : x est statique et accessible partout
        y = y * 2;  // Correct : y est une variable d'instance
        p2();       // Correct : on peut appeler une méthode d'instance depuis une instance
    }

    // Méthode d'instance qui affiche les valeurs de x et y
    void p2() {
        System.out.println("x = " + x + "    y = " + y);
    }

    // Méthode principale (point d'entrée du programme)
    public static void main(String args[]) {
        // Création d'un objet de la classe Class1
        Class1 obj = new Class1();

        // Accès correct à la variable d'instance via l'objet
        obj.y = obj.y * 2;

        // Appel correct de la méthode d'instance via l'objet
        obj.p2();
    }
}
```

Modificateurs avec la méthode main

Modificateur	Accessible dans la classe	Accessible dans le package	Accessible par une sous-classe	Accessible partout
Private	✓	✗	✗	✗
(par défaut)	✓	✓	✗	✗
Protected	✓	✓	✓ (même en dehors du package si hérité)	✗
Public	✓	✓	✓	✓

Opérateurs

Les unités de base du langage Java sont héritées du langage C. Ainsi le langage Java utilise sans modification les entités et structures suivantes du langage C:

	Symbole	Description	Exemple
Opérateurs arithmétiques	-	*soustraction*	*x-y*
	*	*multiplication*	*3 *x*
	/	*division*	*4/2*
	%	*modulo (reste de la division)*	*5%2*
Opérateurs d'affectation	**=**	*Affectation*	***x=2***
	-=	*Soustraction et affectation*	***x-=2***
	+=	*Addition et affectation*	***x+=2***

Opérateurs

	Symbole	Description	Exemple
Opérateurs d'incrémentations et décrémentations	*++*	*Pré-incrémentation*	*++x*
	++	*Post-incrémentation*	*x++*
	--	*Pré-décrémentation*	*--x*
	--	*Post-décrémentation*	*x--*
Opérateurs relationnels	*==*	*égal à*	*x==2*
	<	*inférieur à*	*x<2*
	<=	*inférieur ou égal à*	*x<=3*
	>	*supérieure à*	*x>2*
	>=	*supérieur ou égal à*	*x>=3*
	!=	*différent de*	*a !=b*

Opérateurs

	Symbole	Description	Exemple
Opérateurs logiques	*&&*	*et*	*a && b*
	||	*ou*	*a ||b*
	!	*non*	*!a*

Les conversions d'ajustement de type

Une conversion telle que **int -> float** se nomme une conversion d'ajustement de type. Elle ne peut se faire que suivant une hiérarchie qui permet de ne pas dénaturer la valeur initiale , à savoir : **int -> long -> float -> double**

On peut bien sûr convertir directement un int en double ; en revanche, on ne pourra pas convertir un double en float ou en int.

```
int n     long p      float x

   N    *    p     +  x
   |         |        |
 long        |        |          conversion de n en long
   |         |        |
   |__ *__ |          |          multiplication par p
        |             |
      long            |          le résultat de * est de type long
        |             |
     float            |          il est converti en float
       |___  +  ___|            pour être additionné à x
           float                 ce qui fournit un résultat de type float
```

Les promotions numériques

Les conversions d'ajustement de type ne suffisent pas à régler tous les cas (les opérateurs numériques ne sont pas définis pour les types **byte, char et short**).

Java prévoit tout simplement que toute valeur de l'un de ces deux types apparaissant dans une expression est d'abord convertie **en int**, et cela sans considérer les types des éventuels autres opérandes. On parle alors de **promotions numériques (ou encore de conversions systématiques).**

byte --> short -> int -> long -> float -> double
char -> int -> long -> float -> double

Les promotions numériques

p1, p2 et p3 sont de type short et x de type float,

```
p1    *     p2     +     p3    *    x
|           |            |         |
int         int          int       |     promotions numériques short -> int
|____ * ____|            |         |     multiplication
      |                 float      |     conversion d'ajustement de type
     int                 |___ * ___|
      |                      |
    float                  float         conversion d'ajustement de type
      |_________ + _________|            addition
                |
              float
```

Les promotions numériques

Soit ces déclarations :

```
byte b1 = 10, b2 = 20 ;
short p = 200 ;
int n = 500 ;
long q = 100 ;
float x = 2.5f ;
double y = 5.25 ;
```

Donner le type des expressions arithmétiques suivantes :

```
b1+b2 // 1
p+b1 // 2
b1*b2 // 3
q+p*(b1+b2); // 4
x+q*n // 5
b1*q/x // 6
b1*q*2./x // 7
b1*q*2.f/x // 8
```

Lire les entrées clavier « La classe Scanner »

Scanner sc = new Scanner(System.in);

```
/**
 * @param args
 */
public static void main(String[] args) {
    // TODO Auto-generated method stub

    Scanner sc = new Scanner(System.in);
     System.out.print
     int j = sc.nextI
     System.out.print

}
}
```

Import 'Scanner' (java.util)
Create class 'Scanner'
Change to 'Signer' (java.security)
Rename in file (Ctrl+2, R direct access)

Importer la classe Scanner

import java.util.Scanner;

Un package est un ensemble de classes. En fait, c'est un ensemble de dossiers et de sous-dossiers contenant une ou plusieurs classes.

Lire les entrées clavier « La classe Scanner »

```
//Ceci importe la classe Scanner du package
 import java.util.Scanner;
//Ceci importe toutes les classes du package
 import java.util.*;
```

```
Scanner sc = new Scanner(System.in);
System.out.println("Veuillez saisir un mot :");
String str = sc.nextLine();
System.out.println("Vous avez saisi : " + str);
```

next<Type de variable commençant par une majuscule>

```
Scanner sc = new
Scanner(System.in);
 int i          = sc.nextInt();
double d  = sc.nextDouble();
long l         = sc.nextLong();
byte b       = sc.nextByte();
```

Il y a un type de variables primitives qui n'est pas pris en compte par la classe Scanner : ils'agit du type char.

Lire les entrées clavier « La classe Scanner »

```
System.out.println("Saisissez une lettre :");
Scanner sc   = new Scanner(System.in);
String str     = sc.nextLine();
char carac   = str.charAt(0);
System.out.println("Vous avez saisi le caractère : " + carac);
```

La méthode nextLine()

la méthode **nextLine()** récupère le contenu de toute la ligne saisie et replace la « tête de lecture » au début d'une autre ligne.

```
import java.util.Scanner;

public class Next
 {
      public static void main(String[] args)
       {
          Scanner sc = new Scanner(System.in);
          System.out.println("Saisissez un entier : ");
          int i = sc.nextInt();
          System.out.println("Saisissez une chaîne : ");
          String str = sc.nextLine();
          System.out.println("FIN ! ");
          sc.close();
      }
 }
```

```
//On vide la ligne avant d'en lire une autre
sc.nextLine();
String str = sc.nextLine();
System.out.println("FIN ! ");
```

Fermer le scanner

```
scanner.close();
```

Structures de contrôle

Instructions de choix:

```
if (condition1){
      traitement1;
}

else if (condition2){
       traitement2;
}
            .
            .
else{
       traitementN;
}
```

Classe GreetingExample:

```
if (args.length==1) {
      greet(args[0]);
}
else if (args.length==2) {
       greet(args[0] + "-" + args[1]); }

else if (args.length==3) {
       greet(args[0] + "-" + args[1] +      "-"
+ args[2]);
}

else {
sayHelloTo("world");
}
```

Structures de contrôle

Instructions de choix:

- `java GreetingExample Alice Bob` → **Hello, Alice & Bob!**
- `java GreetingExample Alice Bob Charlie` → **Hello, Alice, Bob, and Charlie!**
- `java GreetingExample` → **Hello, everyone!**
- `java GreetingExample Alice` → **Hello, Alice!**

```
if (args.length==1) {
        greet(args[0]);
}
else if (args.length==2) {
         greet(args[0] + "-" + args[1]); }

else if (args.length==3) {
         greet(args[0] + "-" + args[1] +        "-"     +
args[2]);
}

else {
greet("world");
}
```

Structures de contrôle

Instructions de choix:

switch, **case**, **break**, **default**: Switch utilise ce qu'on appelle des cas, pour comparer une valeur et décider si un bloc de code associé doit être exécuté.

```
switch(variable) {
        case Valeur1: traitement1;
        break;
        case …:
        break;
        case ValeurN: traitement1N;
        break;
        default: traitement;
}
```

```
switch(args.length) {
        case 0: greet("world");
        break;
        case 1: greet(args[0]);
        break;
        case 2: greet(args[0] + "-" +
        args[1]);
        break;
        default: System.out.println(« Vous devez
        fournir au plus 2 arguments!");
}
```

Structures de contrôle

Les boucles

For : Exécute un ensemble d'instructions un nombre de fois fixe, basé sur les valeurs limites inférieure et supérieure de l'énumérateur.

break: Permet de terminer l'exécution d'une boucle (interrompre une séquence d'exécution).

continue: Permet l'interruption d'une itération en cours et retourne au début de la boucle avec exécution de la partie incrémentation (Ignorez quelques instructions à l'intérieur d'une boucle)

```
for (initialisation; condition ; incrémentation ou décrémentation) {
      lesInstruction;
      }
```

Structures de contrôle

Les boucles

```
for ( int i=0; i <10; i++) { // déclarations exécutées à chaque itération
        if(i == 2 ||i == 5) {  continue; }
        System.out.println("Valeur de i : " + i + ".");
}
```

```
int [] myArray = { 10, 20, 30, 40, 50, 60, 70, 80, 90, 100 };
for (int i =0; i<myArray.length;i++) {
        if (myArray[i] == 50) {
                System.out.println ("J'ai trouvé ce que je cherche: " +myArray[i]+ "!");
                break; }
        System.out.println ("Je cherche encore. Je suis arrivé à "+myArray[i]);
}
```

Structures de contrôle

while : Exécute un ensemble d'instructions jusqu'à ce qu'une condition définie soit remplie.

```
while (expression booléenne) {
        instructions;
        }
```

```
int nombreArbres = 0;
while (nombreArbres < 10) {
        nombreArbres += 1;
        System.out.println("J'ai" + nombreArbres +" arbres");
}
```

Structures de contrôle

do, **while**: Elle est très similaire à while, mais la condition est placée à la fin du bloc de code correspondant. De cette façon, le bloc de code sera toujours exécuté au moins une fois.

```
do {
        instructions;
} while (condition);
```

```
int pushUpGoal = 10;
do{
        System.out.println ("Push up " + pushUpGoal);
        pushUpGoal -= 1;
} while(pushUpGoal > 0);
```

3-POO avec java

Objet, Classe, Attributs, Méthodes, Constructeur, Package

Héritage et Accessibilité

Polymorphisme

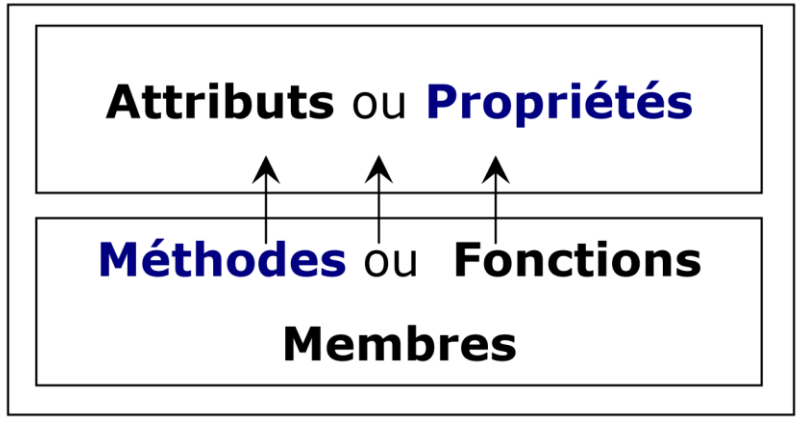
Interfaces / Classes Abstraites

La généricité

Principe de base de la POO

La Programmation Orientée Objets (**POO**) consiste en une structuration de plus haut niveau. Il s'agit de regrouper ensemble les données et toutes les procédures qui permettent la gestion de ces données. On obtient alors des entités comportant à la fois un ensemble de données et une liste de procédures et de fonctions pour manipuler ces données. La structure ainsi obtenue est appelée : **Objet**.

Fichier	
État	nom, répertoire, protections, contenu, ...
Comportement (**Opérations**)	ouvrir, fermer, supprimer,



Principe de base de la POO

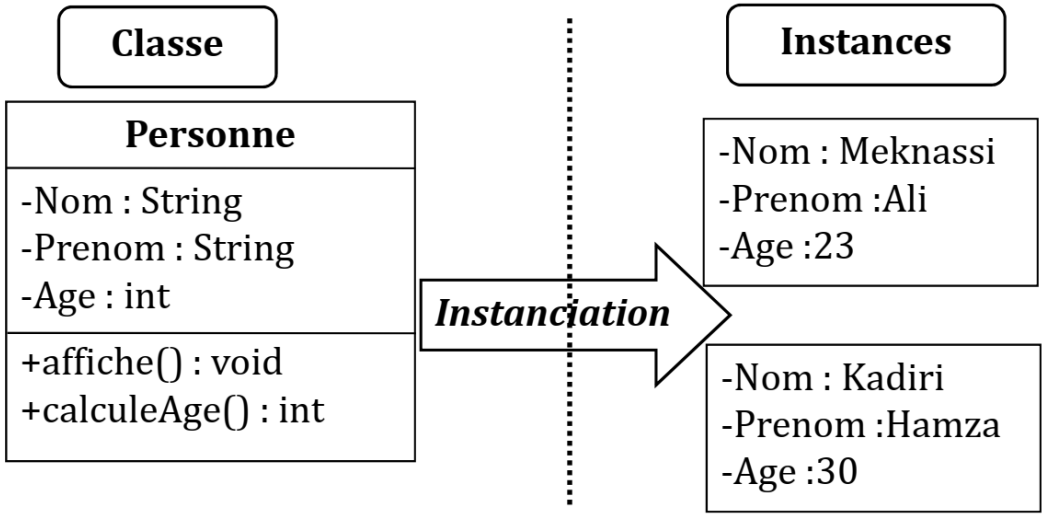
Un objet est alors une généralisation de la notion d'enregistrement. Il est composé de deux parties:

- Une partie statique (fixe) composée de la liste des données de l'objet. On les appelle : **Attributs ou Propriétés**, ou encore : **Données Membres**.
- Une partie dynamique qui décrit le comportement ou les fonctionnalités de l'objet. Elle est constituée de l'ensemble des procédures et des fonctions qui permettent à l'utilisateur de configurer et de manipuler l'objet. Ainsi, les données ne sont généralement pas accessibles directement mais à travers les procédures et les fonctions de l'objet. Celles-ci son appelées: **Méthodes ou Fonctions Membres**.

Encapsulation

Un objet est une variable caractérisée par des propriétés et des méthodes. La définition de ces propriétés et de ces méthodes devra être réalisée dans une structure de données appelée **classe**.
Une **classe** est donc le support de l'encapsulation : c'est un ensemble de données et de fonctions regroupées dans une même entité. Les données seront généralement appelées des propriétés, les fonctions qui opèrent sur ces propriétés sont appelées des méthodes.
Créer un objet depuis une classe est une opération qui s'appelle **l'instanciation**. L'objet ainsi créé pourra être appelé aussi : **instance**. Entre classe et objet il y a, en quelque sorte, le même rapport qu'entre type de données et variable.

Pour utiliser une classe il faut en déclarer une instance de cette classe (appelée aussi objet).



Syntaxe de déclaration d'une classe

```
Class NomDeClass(){
    Propriétés de la classe
    Méthodes de la classe
}
```

Syntaxe de déclaration d'une classe

Exemple d'une classe:

```
public class Point {
      private int x; // abscisse
      private int y; // ordonnee

      public void initialise (int abs, int ord){
          x = abs ;
          y = ord ;
      }

      public void deplace(int dx, int dy) {
          x += dx;
          y += dy;
      }

      public void affiche() {
          System.out.println("Je suis un point de coordonnées " + x + " " + y);
      }
}
```

Les attributs

Les méthodes

Syntaxe de déclaration d'une classe

Méthodes ou fonctions membres: la définition d'une méthode se compose d'un en-tête et d'un bloc (comme les fonctions ordinaires).

```
public void deplace(int dx, int dy)
{
    x += dx;
    y += dy;
}
```

→ **L'entête**

Le bloc

Remarque: L'ordre des méthodes dans une classe **n'a pas d'importance**. Si dans une classe (**Point**), on rencontre d'abord la méthode **affiche()** puis la méthode **deplace(), affiche()** peut être appelée sans problème dans **deplace().**

Déclaration d’un objet

Utilisation d'une classe : Pour pouvoir utiliser une classe il faut d’abord **déclarer un objet**, la déclaration est réalisée de la manière suivante :

```
NomDeClasse NomDeVariable ;
```

Exemple:

```
Point p1; /* cette déclaration ne réserve pas d’emplacement
pour un objet de type Point, mais seulement un emplacement
pour une référence à un objet de type Point.*/
```

Instanciation

Pour **créer une instance** d'une classe, il faut utiliser l'opérateur **new** qui se charge de créer une instance (ou objet) de la classe et de l'associer à la variable.

L'instanciation est réalisée par la façon suivante :

```
NomDeVariable = new NomDeClasse() ;
```

Exemple:

```
p1=new Point(); /*crée un emplacement pour un objet de type
Point et fournit sa référence en p1.*/
```

Syntaxe de déclaration d'une classe

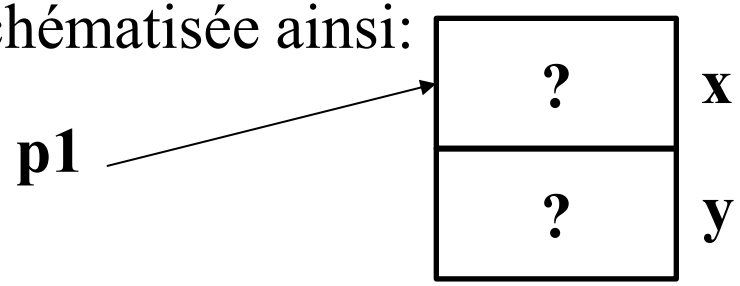
Il est possible de réunir les deux instructions en une seule :

```
NomDeClasse NomDeVariable = new NomDeClasse() ;
```

Exemple:

```
Point p1=new Point(); /*Crée une référence et un emplacement
pour un objet de type Point et fournit sa référence en p1.*/
```

La situation peut être schématisée ainsi:

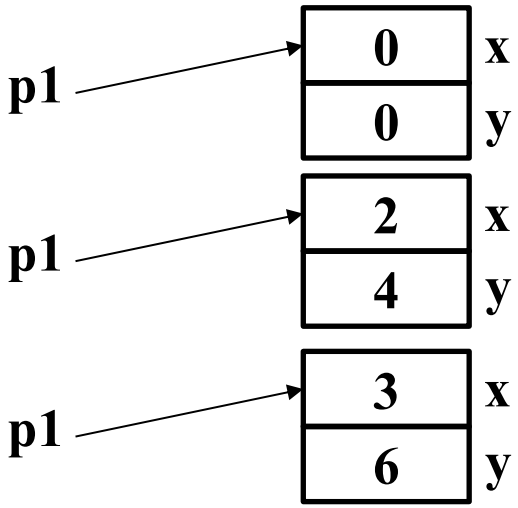


Instanciation

On peut appliquer les méthodes de **Point** à l'objet référencié par **p1**.

Exemple:

```
public static void main(String[] args) {
        Point p1=new Point();
        p1.affiche();
        p1.initialise(2,4);
        p1.affiche();
        p1.deplace(1, 2);
        p1.affiche();
}
```




```
Je suis un point de coordonnées 0 0
Je suis un point de coordonnées 2 4
Je suis un point de coordonnées 3 6
```

Constructeur de classe

Constructeur de classe : Un constructeur est une méthode particulière dans une classe. **Il porte le même nom que la classe**, et permet **d'allouer la mémoire** nécessaire à l'objet et **d'initialiser ses attributs**.

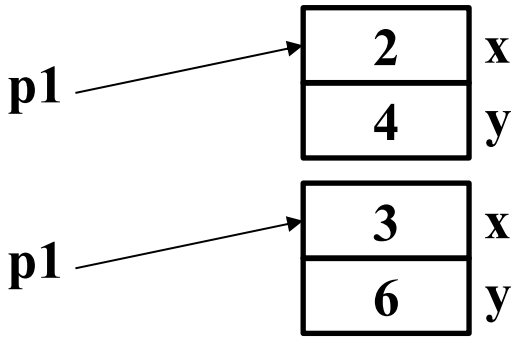
```
public class Point {
      private int x; // abscisse
      private int y; // ordonnee

      public Point (int abs, int ord){  // au lieu de la fonction initialise(..)
           x = abs ;                     // Le constructeur porte le même nom que la classe et il n'a pas un type de retour
           y = ord ;
      }
      public void deplace(int dx, int dy) {
           x += dx;
           y += dy;
      }
      public void affiche() {
           System.out.println("Je suis un point de coordonnées " + x + " " + y);
      }
}
```

Constructeur de classe

Si on ne créer pas de constructeurs de classes, le constructeur sans arguments est là par défaut.

Exemple d'utilisation du constructeur:

```
public static void main(String[] args) {
        Point p1=new Point(2,4);
        p1.affiche();
        p1.deplace(1, 2);
        p1.affiche();
}
```




```
Je suis un point de coordonnées 2 4
Je suis un point de coordonnées 3 6
```

Affectation d'objets

Exemple :

```
NomDeClasse objet1 = new NomDeClasse ();

NomDeClasse objet2 = objet1;
```

Objet1 et objet2 contiennent la même référence et pointent donc tous les deux sur la même espace mémoire: les modifications faites à partir de l'une des deux variables modifient le contenu aussi bien pour l'une que pour l'autre des 2 variables.

Références et comparaison d'objets

Les variables de type objet que l'on déclare ne contiennent pas un objet mais une référence vers cet objet. Lorsque l'on écrit c1 = c2 (c1 et c2 sont des objets), on copie la référence de l'objet c2 dans c1 : c1 et c2 réfèrent donc le même objet (ils pointent sur le même objet). L'opérateur == compare ces références. Deux objets avec des propriétés identiques sont deux objets distincts :

Pour tester l'égalité des contenus de deux instances, il faut munir la classe d'un méthode permettant de réaliser cette opération : la méthode **equals** héritée de Object.

Exemple :

```
Point p1 = new Point (100,50);

Point p2 = new Point (100,50);

if (p1 == p2) { ... } // Faux

if (p1.equals(p2)) { ... } // Vrai
```

Références et comparaison d'objets: Méthode getClass()

Remarque : Pour s'assurer que deux objets sont de la même classe, il faut utiliser la méthode getClass() de la classe Object dont toutes les classes héritent.

Exemple :

```
(obj1.getClass().equals(obj2.getClass()))
```

Durée de vie d'un object

Les objets en java sont tous dynamiques. La durée de vie d'un objet passe par trois étapes :

- la déclaration de l'objet et son instanciation à l'aide de l'opérateur new
- l'utilisation de l'objet en appelant ces méthodes
- la suppression de l'objet : elle est réalisée automatiquement à l'aide du **garbage collector** qui libère l'espace mémoire associé à l'objet ne référence plus cet espace mémoire (par exemple affectation de ***null*** à l'objet) et lorsque cet espace mémoire n'est plus référencée par aucune autre variable).

Il n'existe pas d'instruction ***delete*** comme en C++.

Exemple réel de l'encapsulation

Vous avez un compte bancaire.

- Si votre **variable de solde** est déclarée comme variable **publique** dans le logiciel bancaire, n'importe qui peut connaître le solde de votre compte.
- Si votre variable de solde est déclarée privée, personne ne peut voir le solde de votre compte ➔ Votre compte est sécurisé.
- Si vous voulez voir le solde de votre compte, vous devrez accéder uniquement aux membres privés via des méthodes définies dans cette classe.
- Cette méthode demandera votre nom de titulaire de compte ou votre ID utilisateur et votre mot de passe pour l'authentification.

Le mot clé ‘this’

On utilise le mot clé **this** dans les circonstances suivantes :

1. Ce mot clé sert **à référencer dans une méthode l’instance de l’objet en cours d’utilisation**. **this** est un objet qui est égal à l’instance de l’objet dans lequel il est utilisé.

```
class Voiture {
    String marque;
    Voiture(String marque){
        this.marque = marque;
    }
    void ajouterAuGarage(Garage g) {
        g.ajouterVoiture(this);
    }
}
class Garage{
    void ajouterVoiture(Voiture v) {
        System.out.println("Ajout de la voiture : " + v.marque);
    }
}
public class TestVoiture {
    public static void main(String[] args) {
        Garage g = new Garage();
        Voiture v = new Voiture("Toyota");
        v.ajouterAuGarage(g);
    }
}
```

Le mot clé ‘this’

On utilise le mot clé **this** dans les circonstances suivantes :

2. Dans une méthode, **this** permet **d'éviter les conflits des noms** lorsque les noms d’arguments d’une méthode et les variables locales sont identiques à des noms d’attributs d’une classe.

Exemple :

```
class A {
        private int value;
        public A(int value) {
                value = value;
                this.value = value;
        }
}
```

Remarque: Dans une classe, pour accéder à une propriété « p », on peut écrire **this.p**. Cette notation devient nécessaire dans le cas où une autre variable du contexte (un paramètre ou une variable globale) porte le même nom de la propriété.

Le mot clé ‘this’

On utilise le mot clé **this** dans les circonstances suivantes :

3. Dans les classes définissant plusieurs constructeurs, un constructeur peut invoquer un autre constructeur de cette classe par le mot clé **this().**

```
class Exemple {
   int x;

   Exemple(int x){
      this.x = x;
   }
   Exemple(){
   this(0); // Appelle Exemple (int x) avec x = 0
   }
}
```

Le mot clé ‘this’: Exemple

```
public class Point {
        private int x; // abscisse
        private int y; // ordonnee
        public Point (int abs, int ord){
                x = abs ; y = ord ;
        }
        public Point() {
                // TODO Auto-generated constructor
stub
        }
        public Point doubleCoordonnees (){
                Point resultat=new Point() ;
                resultat.x=2*this.x ;
                resultat.y= 2*this.y ;
                return resultat;
        }
        public void affiche() {
                System.out.println("Je suis un
point de coordonnées " + x + " " + y);
        }
}
```

```
public class Test {

public static void main(String[] args) {

        Point p1=new Point(2,4);
        p1.affiche();
        Point p1Double=p1.doubleCoordonnees();
        p1Double.affiche();

        }
}
```

```
Je suis un point de coordonnées 2 4
Je suis un point de coordonnées 4 8
```

Le mot clé ‘this’: Exemple

```java
public class Point {
        private int x; // abscisse
        private int y; // ordonnee

        public Point (int x, int y){
                this.x = x ;
                this.y = y ;
                /*this.x fait référence à la
variable membre x alors que x fait référence
à la valeur de l'argument.*/
        }

        public void affiche() {
                System.out.println("Je suis un
point de coordonnées " + x + " " + y);
        }
}
```

```java
public class Test {

public static void main(String[] args) {

        Point p1=new Point(2,4);
        p1.affiche();

        }
}
```

```
Je suis un point de coordonnées 2 4
```

Le mot clé ‘this’: Exemple

```
public class Point {
        private int x; // abscisse
        private int y; // ordonnee

        public Point (int x, int y){
                this.x = x ;
                this.y = y ;

System.out.println("Constructeur avec
         2 arguments "+x+" et "+y);
        }

        public Point() {
                this(0,0);
                System.out.println("Constructeur
sans argument");
        }
}
```

```
public class Test {

public static void main(String[] args) {

        Point p0=new Point();
        System.out.println();
        Point p1=new Point(2,4);

        }
}
```

```
Constructeur avec 2 arguments 0 et 0
Constructeur sans argument

Constructeur avec 2 arguments 2 et 4
```

L’opérateur ‘instanceof’

L’opérateur instanceof permet de déterminer la classe de l’objet qui lui est passé en paramètre. La syntaxe est:

```
… Objet instanceof classe …
```

Exemple :

```
void testClasse(Object x) {
        if (x instanceof A ) ...
}
```

Les constantes: mot clé ‘final’

Les constantes sont définies avec le mot clé ***final***: leur valeur ne peut pas être modifiée. Elles sont généralement aussi: ***publique et statique***.

Exemple :

```
public class A {
        public static final double PI = 3.14 ;
}
```

Surcharge (out surdéfinition) des méthodes

La surcharge d'une méthode permet de définir plusieurs fois une même méthode avec des arguments différents. Lors de l'appel à la méthode, le compilateur choisi la méthode qui doit être appelées en fonction du nombre et du type des arguments.

Une méthode est surchargée lorsqu'elle exécute des actions différentes selon le type et le nombre de paramètres transmis.

Surcharge (out surdéfinition) des méthodes

```
public class Number {

        private int value;

        public Number(int value) {

                This.value = value;

        }

        public void inc() {

                value ++;

        }

        public void inc(int step) {

                value = value + step;

        }

}
```

```
Number n = new Number(20) ;

n.inc();      //appellera la première méthode inc() et incrémentera la valeur par
1

n.inc(5);     //appellera la 2ème méthode inc(int) et incrémentera la valeur par
5
```

Les accesseurs

L’encapsulation permet de sécuriser l’accès aux données d’une classe. Ainsi, les donnés déclarées **private** à l’intérieur d’une classe ne peuvent être accédées et modifiées que par des méthodes définies dans la même classe. Si une autre classe veut accéder aux données de la classe, l’opération n’est possible que par l’intermédiaire de méthodes de la classe prévues à cet effet.

Un accesseur est une méthode **publique** qui donne l’accès à une propriété **privée** d’une classe. L’accès peut être réalisé en **lecture ou en écriture**. Par convention, les accesseurs en lecture commencent par le mot ***get*** (ou is ce le type de la propriété est boolean) et sont appelés des **getters** et les accesseurs en écriture commencent par le mot ***set*** et sont appelés des **setters.**

Les accesseurs

```
public class Number {
        private int value;
        public Number(int value) {
                This.value = value;
        }
        public void setValue(int value) {
                this.value = value;
        }
        public int getValue() {
                return value;
        }
}
```

Définition d’un package

En java, il existe un moyen de regrouper des classe voisines ou qui couvrent un même domaine dans une même structure appelée : package. Pour réaliser un package, on écrit un nombre quelconque de classes dans plusieurs fichiers d'un même repertoire et on commence le fichier de classe avec l'instruction suivante :

```
package nom.de.package;
```

Le mot clé package doit être la première instruction dans un fichier source et il ne doit être présent qu'une seule fois dans le fichier source (une classe ne peut pas appartenir à plusieurs packages).
La hiérarchie d'un package se retrouve dans l'arborescence du disque dur puisque chaque package est dans un répertoire nommé du nom du package.

Définition d'un package

- Pour utiliser ensuite le package ainsi créé, on l'importe dans le fichier:

```
import nom.de.package.*;
```

- Pour utiliser une classe **A** particulière du package, on peut utiliser l'instruction suivante:

```
import nom.de.package.A;
```

Remarque: L'utilisation du symbole * donne l'accès à toutes les classes du package mais pas aux sous packages de celui-ci

3-POO avec java

Objet, Classe, Attributs, Méthodes, Constructeur, Package

Héritage et Accessibilité

Polymorphisme

Interfaces / Classes Abstraites

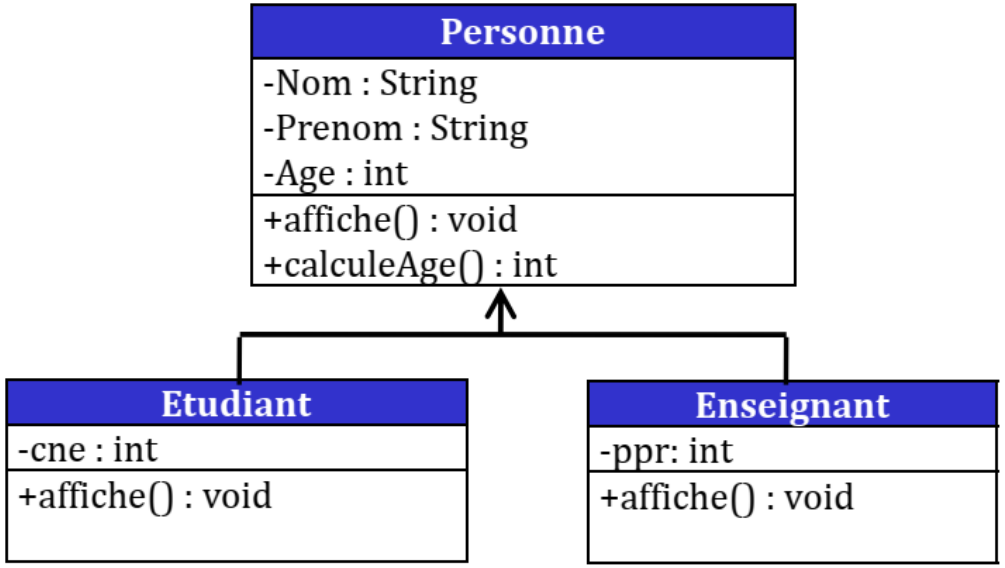
La généricité

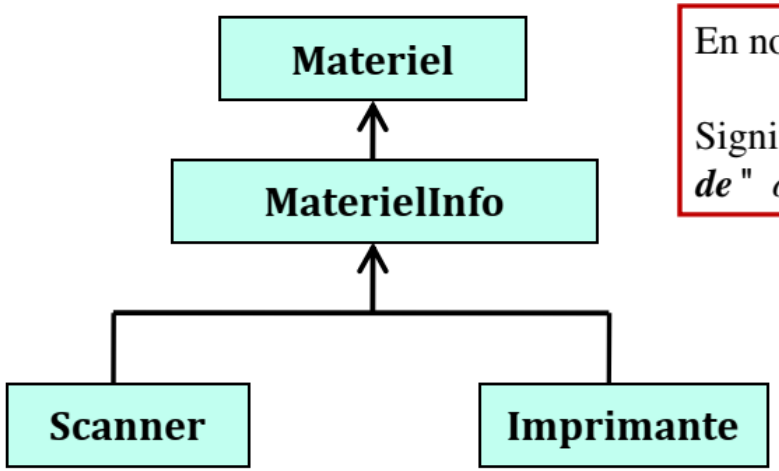
Héritage Java

1. Une classe B **étend** une classe A pour **ajouter ou modifier des services**. On peut aussi ajouter des **propriétés**.
2. La nouvelle classe B est appelée: classe **Fille**, ou classe **Dérivée**.
3. La classe A est la classe **Mère** ou **classe de Base**.
4. On dit que **B étend A** ou **B hérite de A**.
5. L'héritage en Java, ne permet à une classe d'hériter que d'une seule classe de base, on parle d'héritage simple. **En effet, pas d'héritage multiple**.

Héritage Java

- L'héritage induit une relation d'arboresc entre les classes.





En notation UML, le symbole

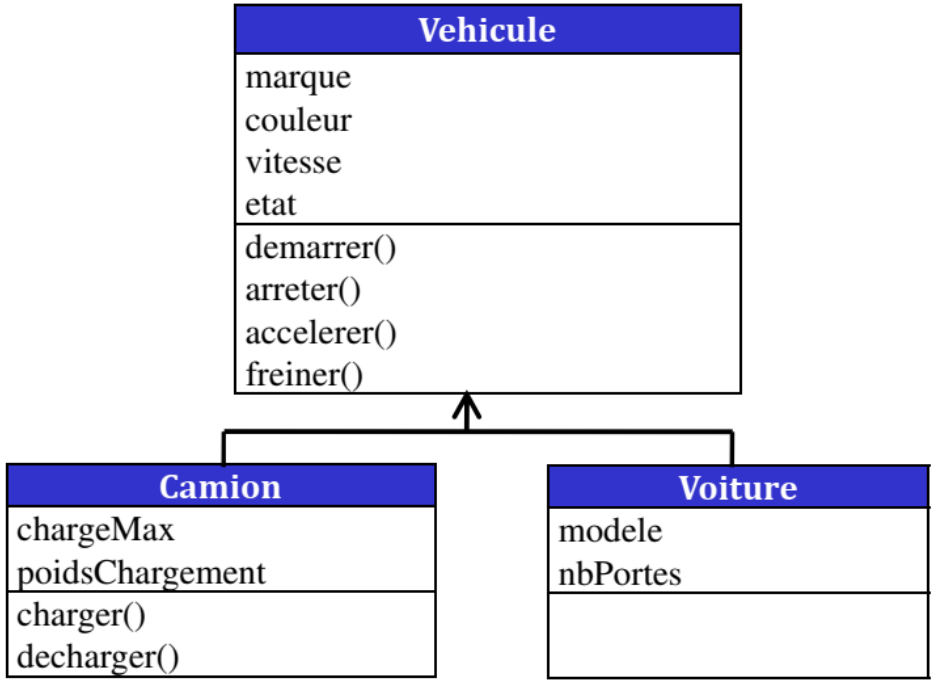
Signifie "***est une sous-classe de***" *ou* "***hérite de***"

«**MaterielInfo**» joue plusieurs rôles

- La classe «**Materiel** » est la classe mère de «**MaterielInfo**».
- La classe «**MaterielInfo**» est la classe mère de «**Scanner**» et «**Imprimante**».
- La classe «**MaterielInfo**» est une sous-classe de «**Materiel** ».
- «**Scanner**» **et** «**Imprimante**» sont des sous-classes de «**MaterielInfo**» .

Héritage Java: Exemple

- A partir de la classe **Vehicule**, de nouvelles classes « **Voiture** » et « **Camion** » peuvent être crées, en utilisant l'héritage.
- Elles la spécialisent en y ajoutant de nouvelles propriétés.
- Un tel diagramme, dans le langage UML, est appelé "Diagramme de classes". Il montre les relations entre les classes d'une application.
- La classe **Camion** est une classe **dérivée** de **Vehicule**. Elle **hérite** tous ses méthodes et attributs. Elle **étend** la classe mère **Vehicule** en lui ajoutant deux **nouveaux attributs** *chargeMax* et *poidsChargement*, ainsi que deux **nouvelles méthodes** *charger()* et *decharger()*).
- La classe **Voiture** est une classe **dérivée** de **Vehicule** et **hérite** toutes les méthodes et les attributs de Vehicule en lui **ajoutant** deux nouveaux attributs *modele* et *nbPortes*.



Héritage Java

Contraintes:

6. Si la classe **A (Mère)** ne dispose d'aucun constructeur (→ le compilateur génère un constructeur sans paramètre pour **A**), la classe **B (Fille)** peut être définie sans contraintes.

Héritage Java

```
Class A {
    private int x, y;
    public void setX(int x){
        this.x = x;
    }
    public int getX(){
        return x;
    }
    public int getY(){
        return y;
    }
    public void print(){
        System.out.println("(" + x + ", " + y
    + ")");
    }

}
```

```
Class B extends A {

}
```

→ **Un constructeur sans paramètre est généré pour B.**

Héritage Java

Contraintes:

6. Si **A** dispose d'un constructeur avec paramètres et pas de constructeur sans paramètres, alors la définition suivante est erronée:

```
Class B extends A {

}
```

En fait un constructeur sans paramètre est généré pour **B**, celui-ci appellera le constructeur sans paramètre de **A** qui n'existe pas. D'où l'erreur.

Héritage Java: Le mot clé ‘super’

8. **Règle Générale:** tous les constructeurs d’une classe fille appellent automatiquement le constructeur sans paramètre de la classe Mère. Cet appel est généré automatiquement par le compilateur comme première instruction de chaque constructeur.

9. Cet appel au constructeur de la classe Mère peut être réalisé explicitement à l’aide du mot clé **super**:

```
Class B extends A {
      B() {
            super();
      }
}
```

Héritage Java: Le mot clé ‘super’

10. Cet appel comme première instruction signifie que si on a une classe C qui dérive de B, alors:

- Le constructeur de A est exécuté en premier lieu
- Ensuite celui de B
- Et enfin celui de C

Héritage Java

```
Class A {
    A(){
        System.out.println("A")
    }
}
```

```
Class B extends A {

    B(){
        System.out.println("B")
    }
}
```

```
Class C extends B {
    C(){
        System.out.println("C")
    }
}
```

Une instanciation de la classe C → l'affichage suivant :

```
A

B

C
```

Héritage Java: Le mot clé ‘super’

11. L’existence de ce mot clé ***super*** permettra de dérouter l’appel à un autre constructeur avec paramètres en précisant entre les parenthèses les arguments d’appel de celui-ci:

```
Class A {
    private int x;

    A(){
        System.out.println("A");
    }
    A(int x){
        this.x = x;
        System.out.println("A(x)");
    }

}

Class B extends A {

    B(){
        super(20);
        System.out.println("B");
    }
}
```

Au résultat, on aura:

```
A(x)

B
```


Université Mohammed V de Rabat

Héritage Java: Le mot clé ‘super’

12. Chaque ***super*** appel la classe Mère immédiatement supérieur.

Héritage Java: Le mot clé ‘super’

```
Class A {
    private int x;

    A(){
        System.out.println("A");
    }
    A(int x){
        this.x = x;
        System.out.println("A(x)");
    }

}

Class B extends A {

    B(){
        super(20);
        System.out.println("B");
    }
}
```

```
Class C extends B {

    C(){
        super(20); // est une erreur
        System.out.println("C");
    }
}
```

Signifie que B a un constructeur avec un paramètre entier, ce qui n’est pas le cas.

Héritage Java: Le mot clé ‘this’

13. **Remarque**: Un constructeur d’une classe peut appeler un autre constructeur de la même classe (pour réutiliser les instructions définies dans ce dernier). On utilisera le mot clé ***this***.

Héritage Java: Le mot clé 'this'

```
class A {
   private int x;
   A() {
         System.out.println("A");
   }
   A(int x) {
         this();
         this.x = x;
         System.out.println("A(x)");
   }
}
```

Ou encore

```
class A {
   private int x;
   A() {
         this(20);
         System.out.println("A");
   }
   A(int x) {
         this.x = x;
         System.out.println("A(x)");
   }
}
```


Université Mohammed V de Rabat

Héritage Java: Le mot clé ‘this’

14. L’instruction ***this(…)*** doit aussi être la toute première instruction du constructeur.

Héritage Java: Ajout des services

15. L’extension d’une classe permettra d’ajouter des services:

Héritage Java: Ajout des services

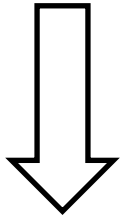
```
Class A {
   A() {
        System.out.println("A");
   }
   void s1() {
        ...
   }
   void s2() {
        ...
   }
}
```

```
Class B extends A {
   B() {
        System.out.println("B");
   }
   void s3() {
        ...
   }
}
```

Une instance 'a' de 'A' aura droit aux services ***s1()*** *et* ***s2().***
Une instance 'b' de 'B' aura droit aux services: ***s1(), s2()*** *(hérités) ainsi qu'au service* ***s3().***

Héritage Java: Ajout des services

Une instance 'a' de 'A' aura droit aux services ***s1()*** *et* ***s2().***
Une instance 'b' de 'B' aura droit aux services: ***s1(), s2()*** *(hérités) ainsi qu'au service* ***s3().***



```
A a = new A() ;

B b = new B() ;

a.s1(); a.s2(); a.s3();

b.s1(); b.s2(); b.s3();
```

Héritage Java: Polymorphisme

16. L'extension d'une classe permettra de modifier des services: On parlera de la ***Redéfinition***:

- Si une méthode est redéfinie dans une classe fille, le **type de retour** doit être **identique** à celui de la méthode correspondante de la classe mère.
- La redéfinition d'une méthode **ne doit pas réduire ses privilèges d'accès.**

private → private, protected, 'sans', public
protected → protected, 'sans', public
'sans' → 'sans', public
public → public

Héritage Java: Polymorphisme

```
Class A {

   A() {

         System.out.println("A");

   }

   void s1() {

         System.out.println("Service s1() de A");

   }

}
```

```
Class B extends A {

   B() {

         System.out.println("B");

   }

   void s1() {

         System.out.println("Service s1() Redéfini dans B");

   }

}
```

```
A a = new A() ;

B b = new B() ;

a.s1(); ➔ "Service s1() de A";

b.s1(); ➔ "Service s1() Redéfini dans B";
```

Héritage Java: Polymorphisme

17. Si on veut réutiliser le service s1() de la classe Mère dans la classe Fille, il est possible d'appeler le service s1() de la classe Mère depuis la fille à l'aide de l'instruction ***super.s1():***

Héritage Java: Polymorphisme

```
Class B extends A {

   B() {

        System.out.println("B");

   }

   void s1() {

        System.out.println("Service s1() Redéfini dans B");
   super.s1();

   }

}
```

```
B b = new B() ;

b.s1(); ➔        "Service s1() Redéfini dans B";

                  "Service s1() de A";
```

Héritage Java

<u>En résumé:</u> Le mot clé super désigne la classe mère. Il peut être utilisé dans deux situations différentes:

1. Pour appeler un constructeur de la classe mère: il est alors utilisé comme première instruction du constructeur de la classe fille:

```
super(paramètres du constructeur de la classe mère);
```

2. Pour appeler une méthode de la classe mère, si celle-ci est redéfinie dans la classe fille.

```
super.nomDeLaMethode(paramètres);
```

Héritage Java: Classe 'Object'

18. **Remarque:** la classe **Object** est la classe mère de toutes les classes sans Mère. Ainsi la définition suivante:

```
class A {

   ...

}
```

Est équivalente à:

```
class A extends Object {

   ...

}
```


Université Mohammed V de Rabat

Héritage Java: Classe 'Object'

Résultats:

- Résultats N°1: la classe **Object** se trouve en tête de l'arbre d'héritage de toutes les classes Java.
- Résultats N°2: Toute nouvelle classe va hériter de toutes les méthodes « public » de la classe **Object**.
- Résultats N°3: comme exemple de méthodes, il y a la méthode toString( ) invoquée automatiquement par le compilateur lors d'une tentative d'affichage d'un objet:

```
System.out.println(objet)
```

⇔

```
System.out.println(objet.toString()) //générée par le compilateur
```

Héritage Java: Classe ‘Object’

Mais aussi dans tout contexte nécessitant cette transformation: généralement les concaténations de chaines de caractères:

```
String s ;

s + objet + … ⇔ s + objet.toString() + …
```

Héritage Java: Classe 'Object'

22. La méthode toString( ) est une méthode **public** de la classe Object qui retourne une chaine de caractère; l'objet est alors « imprimable » ou «affichable ».
23. La méthode toString( ) est définie par défaut (dans Object) pour retourner l'adresse de l'objet sous forme d'une chaine de caractères.
24. Il est donc possible de la redéfinir dans n'importe quelle classe pour choisir le format adéquat d'afficher des objets de la classe.

Exemple :

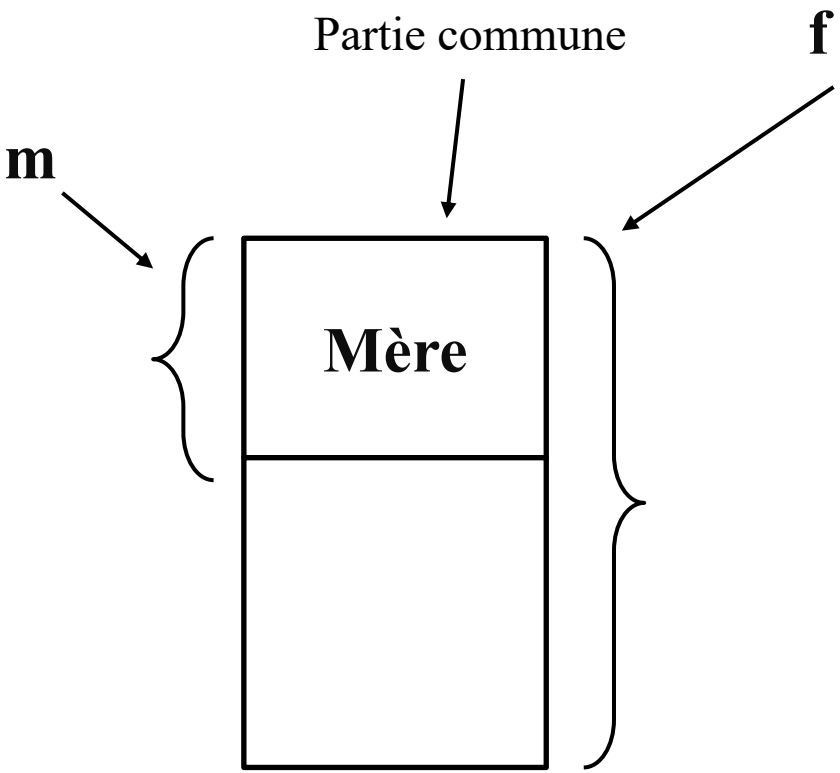
```
class A {

   private int x, y;

   …

   public String toString() {

        return "(" + x + ", " + y + ")") ;

   }

}
```

NECESSAIRE

Transtypage: Upcasting

25. **Important:** un objet de la classe Fille peut être affecté à un objet de classe Mère:
L'affectation **[Mère ← Fille]** est possible (Upcasting):

```
Fille f = new Fille (…) ;
Mère m = f ;
```



→ L'objet « m » aura une vue restreinte que « f ».

Transtypage: Downcasting

26. **Important:** L'affectation **[Mère → Fille]** n'est possible que (Downcasting):
    - Si on utilise un transtypage (Casting)
    - Si l'objet Mère pointe effectivement une Objet Fille:

```
Fille f1 = new Fille(…) ;

Mère m = f1 ; // l'objet mère Pointe une fille.

Fille f2 = (Fille) m ; // Possible
```

```
Mère m2 = new Mère(…) ;

Fille f3 = (Fille)m2 ;// Possible lors de la compilation (syntaxiquement).

                                Mais ERREUR lors de l'Exécution.
```

Transtypage

Remarques:

26. Le casting des objets en Java permet de retrouver l'identité réelle d'un objet.
27. Pas de casting entre des objets de classe n'ayant pas de relation Mère-Fille.
28. Cette relation peut ne pas être directe:

Exemple :

C étend B et B étend A, on peut faire :

```
C c1 = new C(…) ;

A a = c1 ; // Mère ← Fille

C c2 = (C) a; // Fille ← Mère
```

POO avec java

Objet, Classe, Attributs, Méthodes, Constructeur, Package

Héritage et Accessibilité

Polymorphisme

Interfaces / Classes Abstraites

La généricité

3-POO avec java

Objet, Classe, Attributs, Méthodes, Constructeur, Package

Héritage et Accessibilité

Polymorphisme

Interfaces / Classes Abstraites

La généricité

Classes Abstraites et Interfaces

En Java, il existe 3 types d'entités qu'on peut manipuler:

1. Les classes (déjà vues),
2. Les classes abstraites présentées par le mot clé `abstract`,
2. Les interfaces.

Classes Abstraites

Une classe abstraite est une classe qui ne permet pas l'instanciation des objets de son type. C'est une classe incomplète puisqu'elle contient certaines méthodes dont leurs corps sont définies dans la classe dérivée. En effet, elle doit être héritée.
Une classe abstraite comporte une ou plusieurs méthodes abstraites, et elle peut comporter d'autres méthodes non abstraites.
Une méthode abstraite est une méthode qui ne possède pas d'instructions (corps). Elle comporte seulement une signature de définition suivie du caractère point-virgule.
Une méthode abstraite doit être précédée par le mot clé abstract.
Exemple :

Classes Abstraites

Dans une classe abstraite, le corps de quelques méthodes peut ne pas être défini (on déclare

```
abstract class NomClasse {

    abstract type NomMéthode(parameters);

    ...

}
```

Classes Abstraites et Interfaces

En Java, il existe 3 types d'entités qu'on peut manipuler:

1. Les classes (déjà vues),
2. **Les classes abstraites présentées par le mot clé `abstract`,**
3. **Les interfaces.**

Classes Abstraites

Dans une classe abstraite, le corps de quelques méthodes peut ne pas être défini (on déclare uniquement le prototype de la méthode). Ces méthodes sont dites des méthodes abstraites. Une méthode abstraite est aussi présentées par l'intermédiaire du mot clé abstract.
C'est aux classes dérivées de redéfinir ces méthodes et de préciser leur comportement.

Une classe abstraite ne peut donc jamais être instanciée. Il s'agit d'une spécification devant être implémentée par l'intermédiaire d'une classe dérivée. Si cette dernière définit toutes les méthodes abstraites alors celle-ci est instanciable.

```
abstract class NomClasse {

    abstract type NomMéthode(parameters);

    ...

}
```

Classes Abstraites

- Une **classe abstraite** est une classe qui **ne permet pas l'instanciation des objets** de son type.
- C'est une classe incomplète puisqu'elle contient certaines méthodes dont leurs corps sont définies dans la classe dérivée. En effet, **elle doit être héritée.**
- Une classe abstraite **comporte une ou plusieurs méthodes abstraites**, et elle peut comporter d'autres méthodes non abstraites.
- Une méthode abstraite est une méthode qui **ne possède pas d'instructions** (corps). Elle comporte seulement une **signature** de définition suivie du caractère point-virgule.
- Une méthode abstraite doit être précédée par le mot clé ***abstract***.

Classes Abstraites: Exemple

```java
public abstract class Personne {

    public void affiche() {
        System.out.println("Personne");
    } // affiche() est définie dans Personne

    public abstract void determineAge (int annee) ;
    //determineAge() est une méthode abstraite,
    //elle n’est pas définie dans Personne.
}
```

```java
Personne p ;
//on peut déclarer une référence sur un objet
//de type Personne ou dérivé.
p = new Personne (); //Erreur, pas
//d’instanciation d’objets d’une classe
//abstraite.
```

Classes Abstraites: Exemple

Remarque: Une classe abstraite peut ne pas contenir de méthodes abstraites. Cependant, une classe contenant une méthode abstraite doit obligatoirement être déclarée **abstract**.

```
public abstract class Personne {

    public void affiche() {
        System.out.println("Personne");
    } // affiche() est définie dans Personne

    public abstract void determineAge (int annee) ;
    //determineAge() est une méthode abstraite,
    //elle n’est pas définie dans Personne.
}
```

Classes Abstraites: Règles

- Une méthode abstraite **ne peut pas être privée**, ce qui est logique puisque son corps sera définit dans une classe dérivée.
- Les **noms d'arguments muets** doivent figurer dans la définition d'une méthode abstraite.

```
public abstract void determineAge (int); //Erreur : nom argument fictif est obligatoire
```

```
public abstract void determineAge (int annee); //Correct
```

- **Une classe dérivée** d'une classe abstraite **n'est pas obligée de redéfinir** toutes les méthodes abstraites, elle peut ne redéfinir aucune, mais **elle reste abstraite** tant qu'il y a encore des méthodes abstraites non implémentées.
- Une classe dérivée d'une classe **non abstraite** peut être déclarée **abstraite**.

Classes Abstraites: Règles

- Une classe abstraite est **non instanciable** (donc il n’est pas possible d’utiliser les constructeurs de cette classe et donc on ne peut pas créer des objet de son type.)
- Une classe dérivée d'une classe abstraite ne devient **concrète** que si elle implémente toutes les méthodes abstraites de sa classe de base.
- Une méthode abstraite ne possède pas de corps, mais doit être implémentée dans les classes dérivées non abstraites : abstract fonction (arguments); ce n'est qu'une signature .
- Une classe abstraite peut contenir **des méthodes non abstraites** et **des déclarations de variables ordinaires**

Classes Abstraites: Intérêt

- Une classe abstraite peut être **utile** pour **imposer** l'existence de certaines **méthodes** dans des classes, **sans savoir comment elles seront implémentées**.

```
public abstract class Personne {
    public abstract void affiche();
}
```

```
public class Enseignant extends Personne{
    public void affiche() {
        System.out.println("Je suis un
            + "enseignant.");}
}
```

```
public class Etudiant extends Personne{
    public void affiche() {
    System.out.println("Je suis
        + "un étudiant.");}
}
```

- Une classe qui **hérite** d'une classe abstraite **doit redéfinir les méthodes abstraites**, sinon elle est abstraite également.

Classes Abstraites: Intérêt

- Une classe abstraite peut être **utile** pour **imposer** l'existence de certaines **méthodes** dans des classes, **sans savoir comment elles seront implémentées**.

```
public abstract class Animal {
    public abstract void makeSound();
}
```

```
public class Dog extends Animal{
    public void makeSound() {
        System.out.println("Barks");}
}
```

```
public class Cat extends Animal{
    public void affiche() {
    System.out.println("Meows");}
}
```

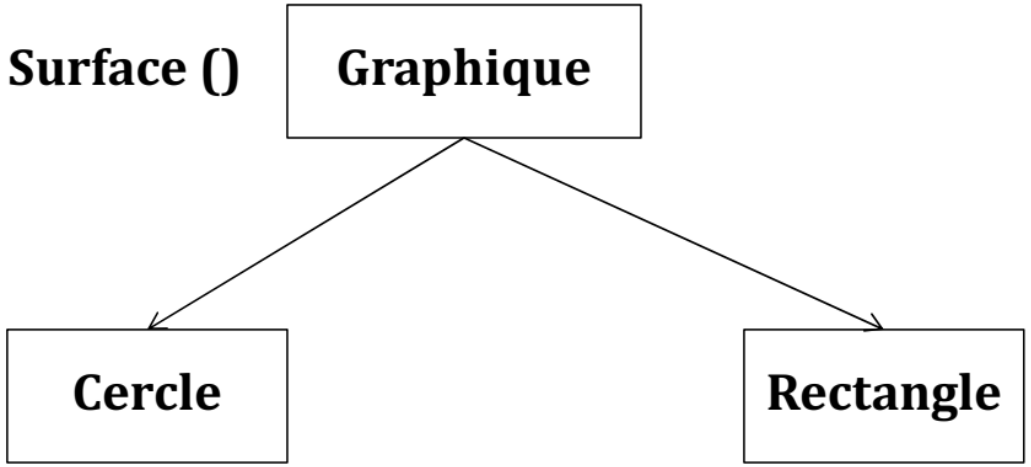
- Une classe qui **hérite** d'une classe abstraite **doit redéfinir les méthodes abstraites**, sinon elle est abstraite également.

Classes Abstraites: Intérêt

- Les classes abstraites facilitent largement la conception orientée objet.
- On peut placer dans une classe abstraite toutes les fonctionnalités dont on souhaite disposer pour les classes descendantes:
  - Soit **sous la forme d'une implémentation complète de méthodes** (non abstraites) et **de champs** lorsqu'ils sont communs à tous les descendants,
  - Soit **sous forme d'interface des méthodes abstraites** dont on est alors sûr qu'elles existeront dans toute classe dérivée instanciable.

Classes Abstraites: Exemple

- Considérons la hiérarchie des classes suivantes :



- On veut que les classes Cercle et Rectangle disposent de la méthode surface() retournant la surface de la figure géométrique définie par la classe.

Classes Abstraites: Exemple

```
public abstract class Graphique {
    private int x, y; //Voilà les champs à hériter

    //Une classe abstraite ne peut pas être instanciée, mais elle peut avoir
    //un constructeur :
    public Graphique ( int x, int y) {
        this.x = x ; this.y = y ;}

    //Voilà la méthode abstraite à compléter :
    public abstract double surface();

    //Il peut y avoir des méthodes non abstraites
    public void affiche() {
        System.out.println("Le centre de l’objet se trouve dans :" + x + " et " + y);
        System.out.println("La surface de l'objet est :" + this.surface());}
}
```

Classes Abstraites: Exemple

```java
public class Cercle extends Graphique {
    private double rayon;

    public Cercle(int x, int y, double r) {
        super(x, y);
        rayon = r;}

    public double surface() {
        return (2 * 3.14 * rayon);}
}
```

```java
public class Rectangle extends Graphique {
    private int largeur, longueur;

    public Rectangle(int x, int y, int la, int lo){
        super(x, y);
        this.longueur = lo;
        this.largeur = la;}

    public double surface() {
        return (longueur * largeur);}
}
```

Classes Abstraites: Exemple

```
public class TestGraphique {
    public static void main(String[] args) {
        Graphique[] tab = new Graphique[6];
        tab[0] = new Graphique (3,2);
        tab[1] = new Cercle(10, 7, 3);
        tab[2] = new Rectangle(4, 7, 8, 6);
        tab[3] = new Rectangle(8, 10, 12, 10);
        tab[4] = new Cercle(8, 5, 3);
        tab[5] = new Rectangle(10, 17, 3, 8);
        for (int i = 1; i <= 5; i++) {
            tab[i].affiche();
        // Y-a-t-il des erreurs?
        // Quel est le résultat?
        }}
}
```

tab[0] = new Graphique (3,2);
Erreur, classe abstraite est non instanciable.
Il est tout de même possible de déclarer des variables de ce type qui pourront recevoir des objets créés à partir des sous-classes concrètes

```
Le centre de l’objet se trouve dans :10 et 7
La surface de l'objet est :18.84
Le centre de l’objet se trouve dans :4 et 7
La surface de l'objet est :48.0
Le centre de l’objet se trouve dans :8 et 10
La surface de l'objet est :120.0
Le centre de l’objet se trouve dans :8 et 5
La surface de l'objet est :18.84
Le centre de l’objet se trouve dans :10 et 17
La surface de l'objet est :24.0
```

Interfaces: Concept

- Une **interface** est un ensemble d'opérations utilisées pour spécifier un **service offert** par une classe.
- Elle peut être vue comme une classe **sans attributs** (**sauf les constantes**) et dont toutes les opérations sont spécifiées mais **pas définies** à priori.
- Elle peut être vue comme **un contrat** ou **un modèle** que doivent offrir toutes les classes qui se réclament (implémentent) de cette interface.

```
public interface Polygone {
    int surface();
    void deplace(int x, int y);
}
```

Interfaces vs Classes Abstraites

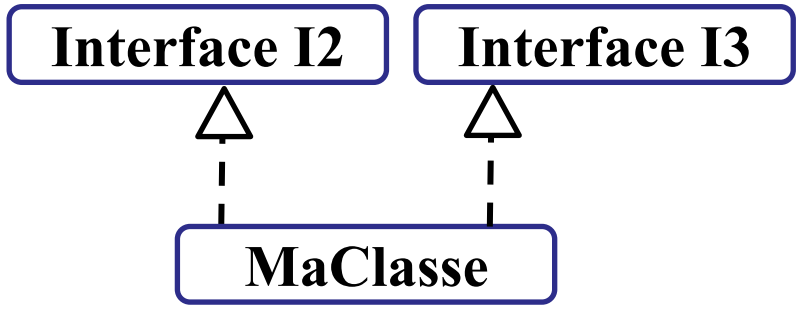
Le corps d'une interface ressemble beaucoup à celui d'une classe abstraite avec cependant les différences suivantes :

- Une interface ne peut définir que des méthodes abstraites. Le modificateur **abstract n'est pas nécessaire**.
- Toutes les méthodes abstraites sont **implicitement publiques** même si le modificateur public est omis.
- Une interface **ne peut pas déclarer des méthodes statiques comme abstraites**.
- Une interface **ne peut pas définir de champs d'instance**.
- Une interface peut cependant contenir **des constantes déclarées static et final** (tous les champs sont **implicitement static et final** même si les modificateurs correspondants sont omis).
- Une interface **ne définit pas de constructeur** (on ne peut pas l'instancier et elle n'intervient pas dans le chaînage des constructeurs).

Interfaces vs Classes Abstraites

- Si l'on considère une classe abstraite n'implémentant aucune méthode et aucun champ (sauf les constantes), on aboutit à la notion d'interface.
- **L'interface est plus riche** qu'un simple cas particulier des classes abstraites :
  - **Une classe peut implémenter plusieurs interfaces** (alors qu'une classe ne pouvait dériver que d'une seule classe abstraite).

```
public class MaClasse implements I1, I2{}
```

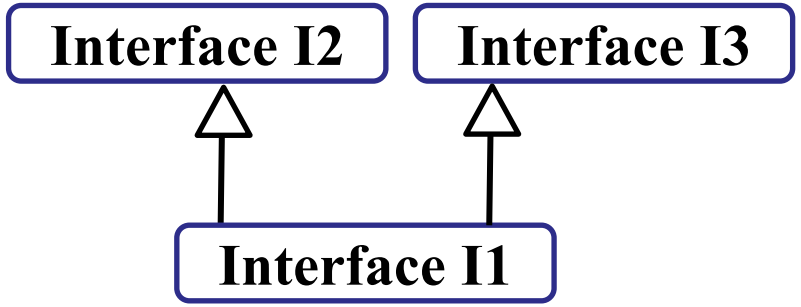


```
public class MaClasse extends C1, C2{} // Pas possible
```

Interfaces vs Classes Abstraites

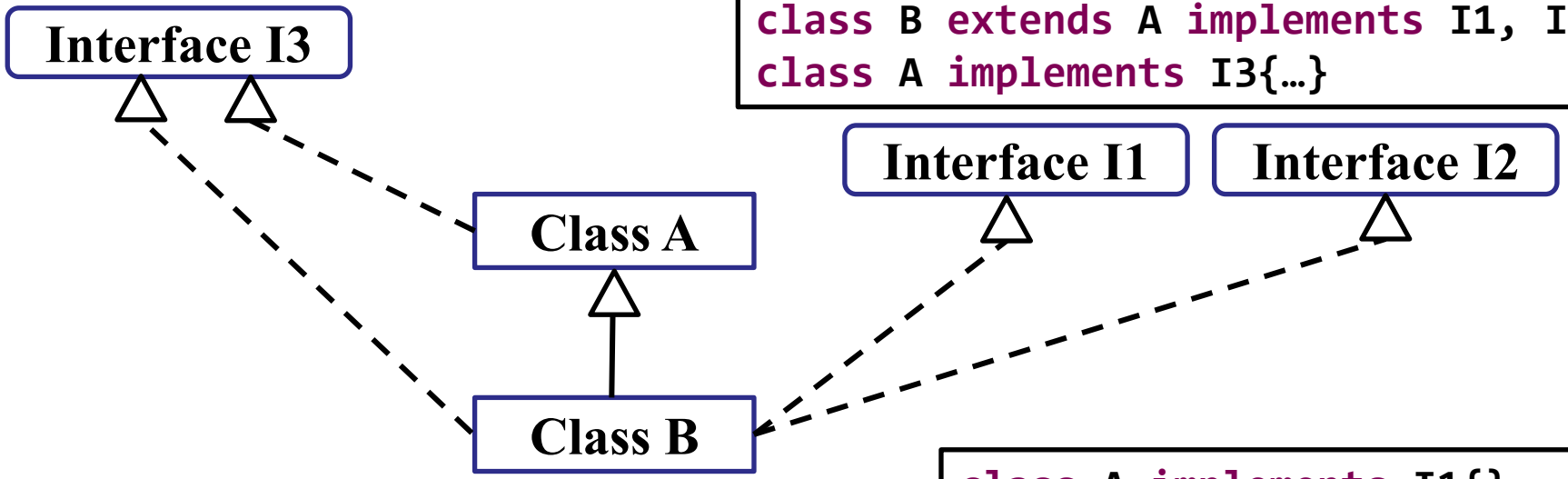
- **L'interface est plus riche** qu'un simple cas particulier des classes abstraites :
  - Les interfaces peuvent se dériver et même utiliser l'héritage multiple:

```
public interface I1 extends I2, I3{……}
```



Interfaces vs Classes Abstraites

- **L'interface est plus riche** qu'un simple cas particulier des classes abstraites :
  - Une classe dérivée peut implémenter 1 ou plusieurs interfaces, elle peut même ré-implémenter une interface déjà implémentée par la superclasse.



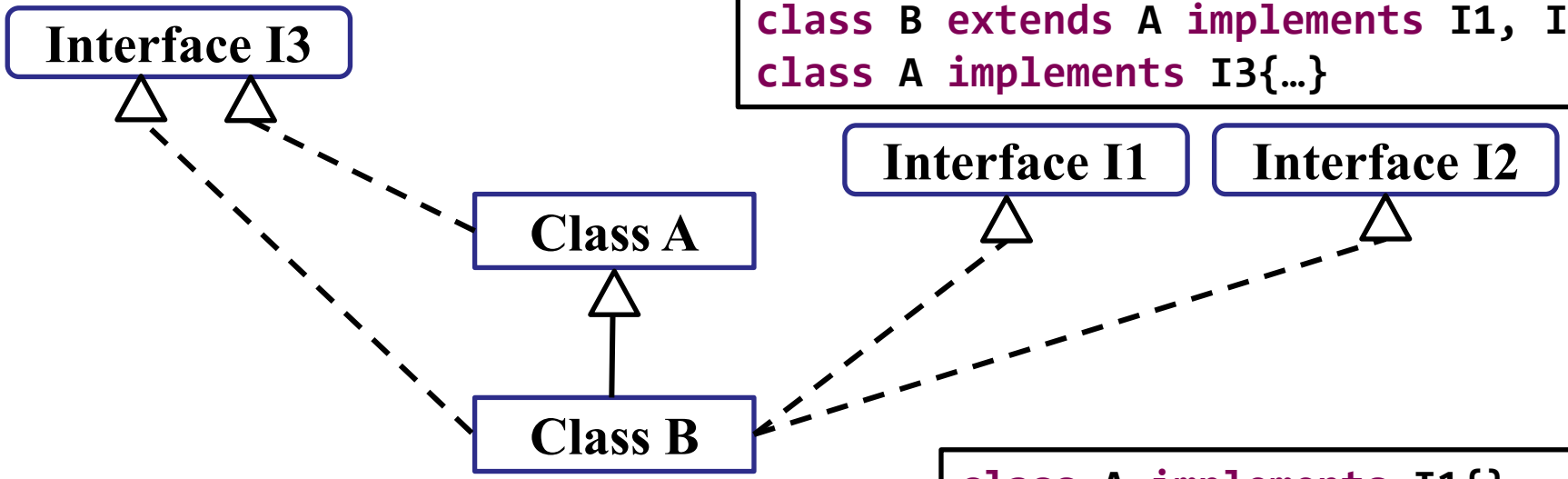

```
class B extends A implements I1, I2, I3 {…}
class A implements I3{…}
```

  - On pourra utiliser des variables de type interface.

```
class A implements I1{}
I1 obj=new A();
```

Interfaces vs Classes Abstraites

- **L'interface est plus riche** qu'un simple cas particulier des classes abstraites :
  - Une classe dérivée peut implémenter 1 ou plusieurs interfaces, elle peut même ré-implémenter une interface déjà implémentée par la superclasse.




```
class B extends A implements I1, I2, I3 {…}
class A implements I3{…}
```

  - On pourra utiliser des variables de type interface.

```
class A implements I1{}
I1 obj=new A();
```

Interfaces vs Classes Abstraites

```java
public interface Polygone {
    int max = 100;

    double surface();
    void deplace(int x, int y);}
```

```java
public interface Polygone {
    public static final int max = 100;

    public abstract double surface();
    public abstract void deplace(int x, int y);}
```

Une interface peut renfermer aussi des constantes

```java
public class Cercle implements Affichable, Polygone {
    private double rayon;
    private Point centre;

    public double surface() {
        return (double) (2 * 3.14 * rayon);}
    public void deplace(int x, int y) {
        centre.setX(centre.getX()+x);
        centre.setY(centre.getY()+y);}
    public void afficher() {
        System.out.println("Je suis un cercle");}
}
```

```java
public interface Affichable {
    void afficher();
}
```

Lorsqu'on définit une classe, on peut préciser qu'elle implémente une ou plusieurs interface en utilisant le mot clé « *implements* »

Interfaces et dérivation

- L’interface est totalement indépendante de l’héritage : Une classe dérivée peut implémenter une ou plusieurs interfaces.
- L'implémentation d'une ou plusieurs interfaces (implements) peut être combinée avec l'héritage simple (extends).
- La clause implements doit suivre la clause extends.

```
public class Cercle extends Graphique implements Affichable, Polygone
```

Interfaces: Caractéristiques

- Lorsqu'une classe déclare une interface dans sa clause implements, elle indique ainsi qu'elle s'engage à fournir une implémentation (c'est-à-dire un corps) pour chacune des méthodes de cette interface.
- Si une classe implémente une interface mais ne fournit pas d'implémentation pour toutes les méthodes de l'interface, c-à-d elle hérite des méthodes (abstraites) non implémentées , elle doit être déclarée abstract.
- Si une classe implémente plus d'une interface, elle doit implémenter toutes les méthodes de chacune des interfaces mentionnées dans la clause implements (ou alors doit être déclarée abstract).

Interfaces vs Classes Abstraites

```
public interface Polygone {
    int max = 100;

    int surface();
    void deplace(int x, int y);}
```

```
public interface Affichable {
    void afficher();
}
```

La classe Carre est déclarée abstract car elle n'implémente pas toutes les méthodes des interfaces implémentée

Toutes les méthodes abstraites sont implicitement publique: Pas besoin du mot clé *public*

```
public abstract class Carre implements Affichable, Polygone{

    abstract double surface();
    abstract void deplace(int x, int y) ;

    public void afficher() {
        System.out.println("Je suis un carré.");}
}
```

Interfaces: Conflits des noms

```
interface I1 {
    void f(int n) ;
    void g();
}
```

```
interface I2 {
    void f(float x);
    void g();
}
```

```
class A implements I1, I2{
    void f(int n) {}
    void f(float x) {}
    void g();
}
```

A doit implémenter la méthode g() existante dans les 2 interfaces une seule fois

---

```
interface I1 {
    void f(int n) ;
    void g();
}
```

```
interface I2 {
    void f(float x);
    int g();
}
```

la méthode « g () » est présentée de 2 façons différentes dans I1 et I2

```
class A implements I1, I2{
    void f(int n) {}
    void f(float x) {}
    void g();
    int g();
}
```

Erreur car void g() et int g() ne peuvent pas coexister au sein de la même classe

Méthodes génériques

- Les méthodes génériques sont celles qui sont écrites avec une seule déclaration et qui peuvent être appelées avec des arguments de différents types.

- En fonction des types d'arguments passés à la méthode générique, le compilateur gère chaque appel de méthode de manière appropriée.

```
public static <E> void  genericDisplay(E element){
    System.out.println(element.getClass().getName()
    +"="+element);
}
```

```
String str = "Type String";
Carre.genericDisplay(str);

Integer i = 25;
Carre.genericDisplay(i);
```

```
java.lang.String=Type String
java.lang.Integer=25
```

Méthodes génériques

- Voici les règles pour définir les méthodes génériques:
  - Les méthodes génériques ont **un type paramétré** (qui représente un paramètre de **type générique**) **avant le type de retour** de la déclaration de méthode.

```
public static <E> void  genericDisplay(E element){
    System.out.println(element.getClass().getName()
    +"="+element);}
```

  - Les méthodes génériques **peuvent avoir différents types paramétrés séparés par des virgules** dans la signature de méthode.

```
public static <E,T> void  affichage2(E element1, T element2){
    System.out.println("element 1= "+element1+ ", element2=
    "+element2);}
```

```
affichage2(str,i);
```

```
element 1= Type String, element2= 25
```

Méthodes génériques

Type paramétré

- Les conventions de dénomination de types paramétrés sont importantes pour apprendre la généricité en Java. Ils doivent contenir une seule lettre en majuscule. Les types paramétrés courants sont les suivants: **T – Type, E – Element, K – Key, N – Number, V – Value**.
- Le type T, par exemple, indique que la méthode peut faire référence à n'importe quel type objet (comme String, Integer et Personne) et non pas un type de base.
- On peut utiliser des types spécifiques héritant d'une classe spécifique. Exemple:

<N extends Number>, < T extends ClassA >

Méthodes génériques: Exemple

```java
public class Main {
   // méthode générique
   public static <E> void display(E[] tab) {
      for(E e : tab) {
         System.out.printf("%s ", e);
      }
      System.out.println();
   }
   public static void main(String args[]) {
      Integer[] i = {1, 2, 3};
      System.out.println("Tableau des entiers:");
      display(i);   // passer un tableau entier
      Character[] c = {'A', 'B', 'C'};
      System.out.println("\nTableau des caractères:");
      display(c);   // passer un tableau de caractères
   }
}
```

```
Tableau des entiers:
1 2 3

Tableau des caractères:
A B C
```

Classes génériques

- La déclaration d'une classe générique ressemble à une déclaration de classe non générique, sauf que le nom de la classe est suivi du type paramétré. Exemple:

```
class MaClasse <T> {}
```

- Comme pour les méthodes génériques, la section des paramètres d'une classe générique peut avoir un ou plusieurs paramètres de type séparés par des virgules.

```
class MaClasse <T,E> {}
```

- Ces classes sont appelées classes paramétrées ou types paramétrés car elles acceptent un ou plusieurs paramètres.
- Une classe qui peut faire référence à n'importe quel type est appelée une classe générique.

Classes génériques: Exemple

```java
class MaClasse <T> {
  T obj;

  void setObj(T obj){
    this.obj = obj;
  }

  T getObj(){
    return obj;
  }
}
```

```java
public class TestGen {

    public static void main(String args[]) {
        MaClasse<Integer> obj = new MaClasse<Integer>();
        obj.setObj(5);
        //obj.setObj("String"); // Erreur de compilation
        System.out.println(obj.getObj());
    }
}
```

```
5
```

Interfaces génériques

- La généricité fonctionnent également avec les interfaces.
- Les interfaces génériques sont spécifiées tout comme les classes génériques.

```
interface MyInterface<T> { ... }
```

- Si une classe implémente une interface générique, on doit indiquer les types des paramètres avec lesquels la classe fonctionne.

```
class Myclass implements MyInterface<Integer> {...} //On a spécifié la param à Integer;
```

- Sinon, cette classe est aussi générique.

```
class Myclass<T> implements MyInterface<T> {...}
```

L'interface Comparable

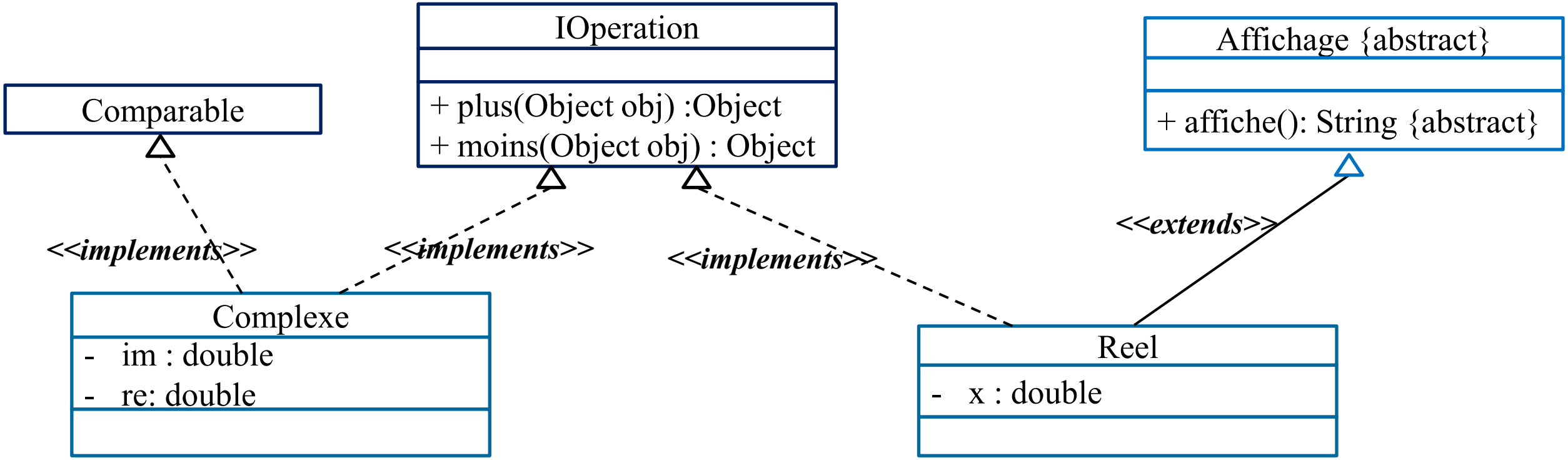
- **Comparable** est une interface générique définie dans l'API Java (package **java.lang**). Elle impose aux classes qui vont l'implémenter la définition de la méthode compareTo():

```
public interface Comparable<T> {
    public int compareTo(T o);
}
```

- La méthode ***compareTo()*** permet de comparer l'objet courant (this) à l'objet o reçu en argument et de renvoyer un entier (dont la valeur exacte est sans importance) :
  - **négatif** (la valeur -1) si l'on considère que l'objet courant est « **inférieur** » à l'objet o (au sens de l'ordre qu'on veut définir),
  - **nul** (la valeur 0) si l'on considère que l'objet courant **est égal** à l'objet o (il n'est ni inférieur, ni supérieur),
  - **positif** (la valeur 1) si l'on considère que l'objet courant est « **supérieur** » à l'objet o

Exercice

- Implémentez le code correspondant au diagramme de classes suivant et donnez un programme de test.



Exercice: Corrigé

```java
public class Reel extends Affichage implements IOperation<Reel>{
    private double x;

    public Reel(double x) {
        this.x = x;}

    public String affiche() {
        return Double.toString(x);}

    public Reel plus(Reel r1) {
        Reel resultat= new Reel(this.x+r1.x);
        return  resultat;}

    public Reel moins(Reel r1) {
        Reel resultat= new Reel(this.x-r1.x);
        return  resultat;}}
```

```java
public interface IOperation <T>{
    public T plus(T nombre);
    public T moins(T nombre);
}
```

```java
public abstract class Affichage {
    public abstract String affiche();
}
```

Exercice: Corrigé

```java
public class Complexe implements IOperation<Complexe>, Comparable<Complexe>{

private double im, re;

    public Complexe(double im, double re) {
        this.im = im;
        this.re = re;}

    public Complexe plus(Complexe nombre) {
        Complexe resultat=new Complexe(this.im+nombre.im, this.re+nombre.re);
        return resultat;}

    public Complexe moins(Complexe nombre) {
        Complexe resultat=new Complexe(this.im-nombre.im, this.re-nombre.re);
        return resultat;}

    public String toString() {
        return "Complexe [im=" + im + ", re=" + re + "]";}
}
```

```java
public int compareTo(Complexe o) {
    if(this.module()==o.module())
        return 0;
    if(this.module()<o.module())
        return -1;
    else
        return 1;}
```

```java
public double module() {
    return Math.sqrt(im*im+re*re);}
```

Exercice: Corrigé

```java
Complexe c1=new Complexe(1,2);
Complexe c2=new Complexe(2,3);

System.out.println(c1);
System.out.println(c2);


Reel r1=new Reel(1);
Reel r2=new Reel(2);

System.out.println(r1.affiche());
System.out.println(r2.affiche());

System.out.println(c1.plus(c2));

System.out.println(r1.moins(r2).affiche());
```

```java
int a=c1.compareTo(c2);
switch(a) {
    case -1: System.out.println(c1.toString()+"
    est inférieur à "+c2.toString());
    break;
    case 0: System.out.println(c1.toString()+"
    est égal à "+c2.toString());
    break;
    case 1: System.out.println(c1.toString()+"
    est supérieur à "+c2.toString());
    break;
}
```

```
Complexe [im=1.0, re=2.0]
Complexe [im=2.0, re=3.0]
1.0
2.0
Complexe [im=3.0, re=5.0]
-1.0
Complexe [im=1.0, re=2.0] est inférieur à
Complexe [im=2.0, re=3.0]
```